

PRAXISSEMINAR · 3 TAGE

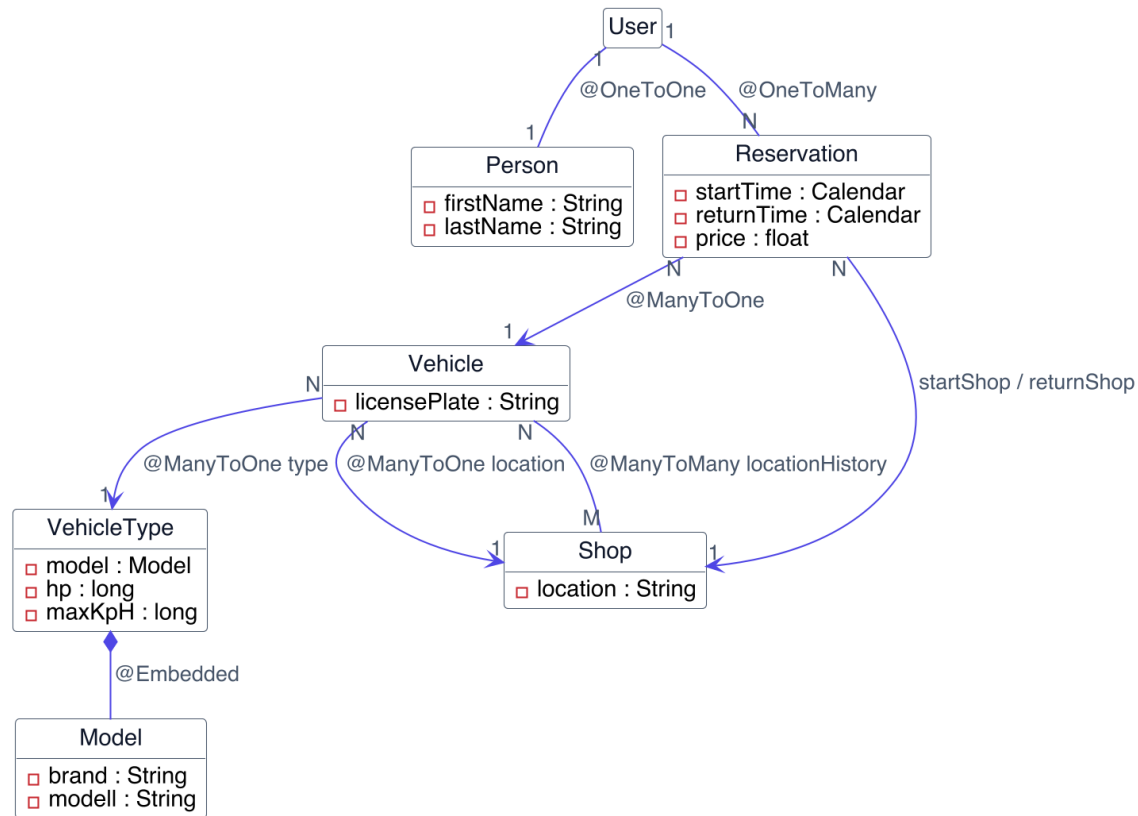
Jakarta Persistence & Hibernate

Objektrelationales Mapping in der Praxis – vom ersten Entity bis zur Criteria API

Rent-a-Car · 40 Übungsmodule · Hibernate 6 · Jakarta Persistence 3.x

Unsere Domäne: Rent-a-Car

Ein Modell, das uns vier Tage lang begleitet



- Alle Übungen und alle Codebeispiele verwenden dieses Modell.
- **VehicleType** beschreibt den Typ, **Vehicle** das konkrete Fahrzeug.
- **Shop** ist die Filiale: aktueller Standort (`@ManyToOne`) und Historie (`@ManyToMany`).
- **Reservation** verbindet Fahrzeug, Start- und Rückgabefiliale.
- **User** ist der Zugang, **Person** die natürliche Person dahinter.
- Quelle: Modul 2_00_Model4QueryProjects

Der Weg durch die drei Tage

Fortschrittsanker – diese Folie kommt zu Beginn jedes Tages wieder

Tag	Schwerpunkt	Module	Kernfrage
Tag 1	Konfiguration, Entities, IDs, Converter, Embeddables	1_0 – 1_4	Wie wird aus einer Klasse eine Tabelle?
Tag 2	Assoziationen, Kaskadierung, Vererbung	1_5 – 1_9_4	Wie bilde ich Beziehungen ab?
Tag 2-3	JPQL, Joins, Projektionen, Fetch-Strategien	2_00 – 2_11	Wie frage ich effizient ab?
Tag 3	Transaktionen, Locking, Lifecycle, Criteria API	2_08_01 – 2_15	Wie bleibt es unter Last korrekt?

Roter Faden: Mapping → Assoziationen → Vererbung → JPQL → Transaktionen → Criteria API

Ihr Material – und wann Sie es brauchen

Drei Bausteine, drei unterschiedliche Zeitpunkte

Cheat Sheet

Alle Annotationen, JPQL-Syntax und Lifecycle-Regeln auf wenigen Seiten.

Ausgabe: heute zu Beginn, als gedrucktes Handout.

Datei: JPA_Cheat_Sheet.pptx

Troubleshooting & Best Practices

LazyInitializationException, N+1, detached entities, equals/hashCode.

Eigene 45-Minuten-Session am Ende von Tag 3
– genau dann, wenn Sie die Fehler selbst erlebt haben.

Datei:
JPA_Troubleshooting_und_Best_Practices.pptx

Übungsprojekte

40 Maven-Module, je Aufgabe ein Modul und ein Lösungsmodul.

`mvn test` – Aufgabe prüfen

`mvn -Pinspect test` – SQL mitlesen

AUFGABE.md in jedem Modul

01

Grundlagen & Mapping

Nach diesem Kapitel können Sie ...

- erklären, welches Problem ORM löst – und welches nicht,
- eine Persistence Unit für Hibernate korrekt konfigurieren,
- für einen konkreten Fall die passende ID-Strategie begründet wählen,
- Werte über AttributeConverter und Embeddables abbilden.

Warum ORM?

JDBC – Sie schreiben das Mapping selbst

```
PreparedStatement ps = con.prepareStatement(
    "SELECT id, hp, brand FROM vehicletype WHERE id = ?");
ps.setLong(1, id);
ResultSet rs = ps.executeQuery();
VehicleType vt = null;
if (rs.next()) {
    vt = new VehicleType();
    vt.setId(rs.getLong("id"));
    vt.setHp(rs.getLong("hp"));
}
rs.close(); ps.close();
// Änderung speichern? Noch einmal 12 Zeilen UPDATE.
```

JPA – das Mapping ist deklariert

```
VehicleType vt = em.find(VehicleType.class, id);

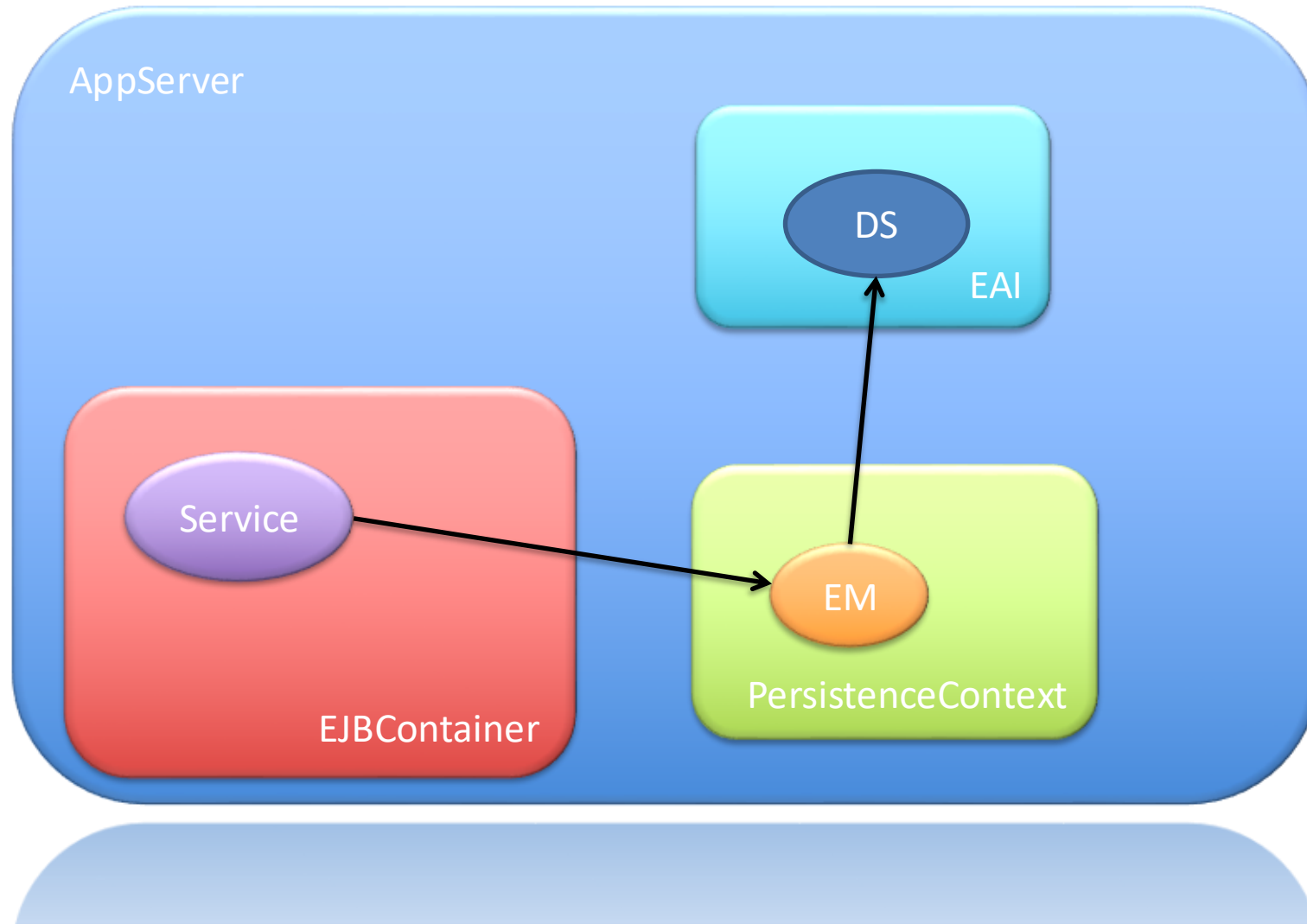
vt.setHp(150);
// kein UPDATE nötig: Dirty Checking beim Commit

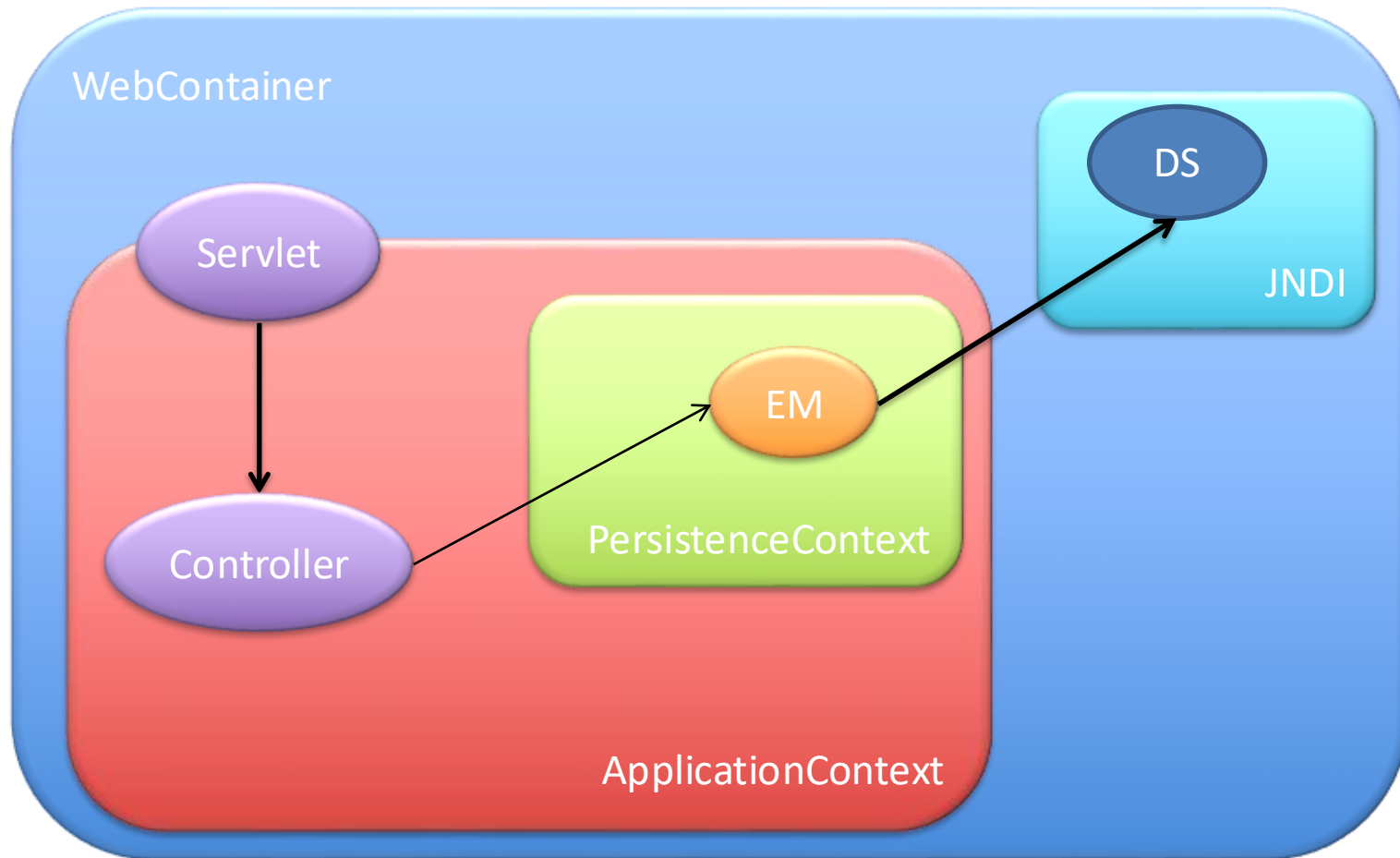
// Identität garantiert:
em.find(VehicleType.class, id) == vt // true

// Beziehungen folgen dem Objektgraph:
vt.getVehicles().size();
```

ORM spart nicht Tipparbeit – es verlagert Identität, Zustandsverwaltung und Transaktionsgrenzen in den Persistence Context.

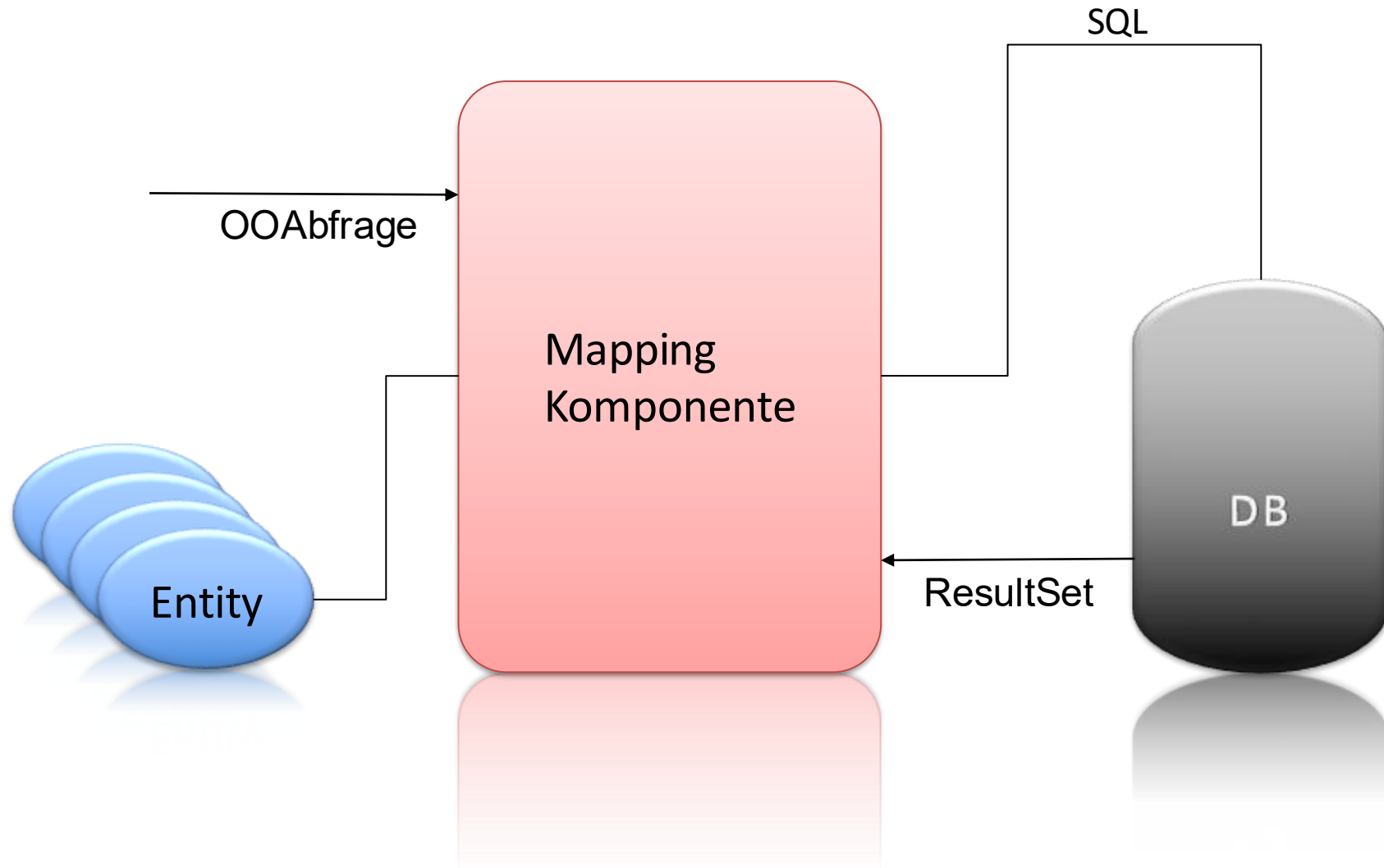
JEE World



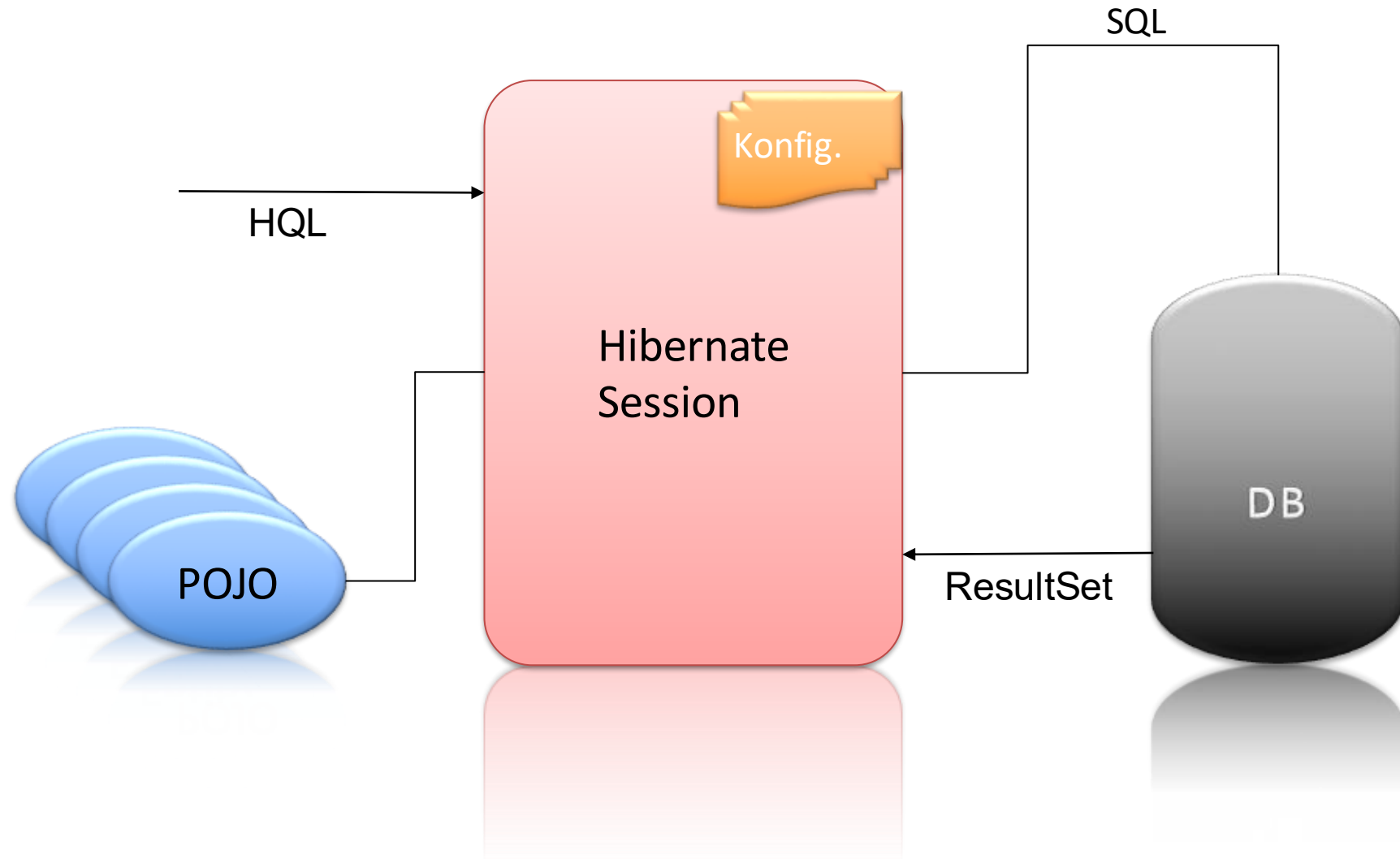


Spring on WEB

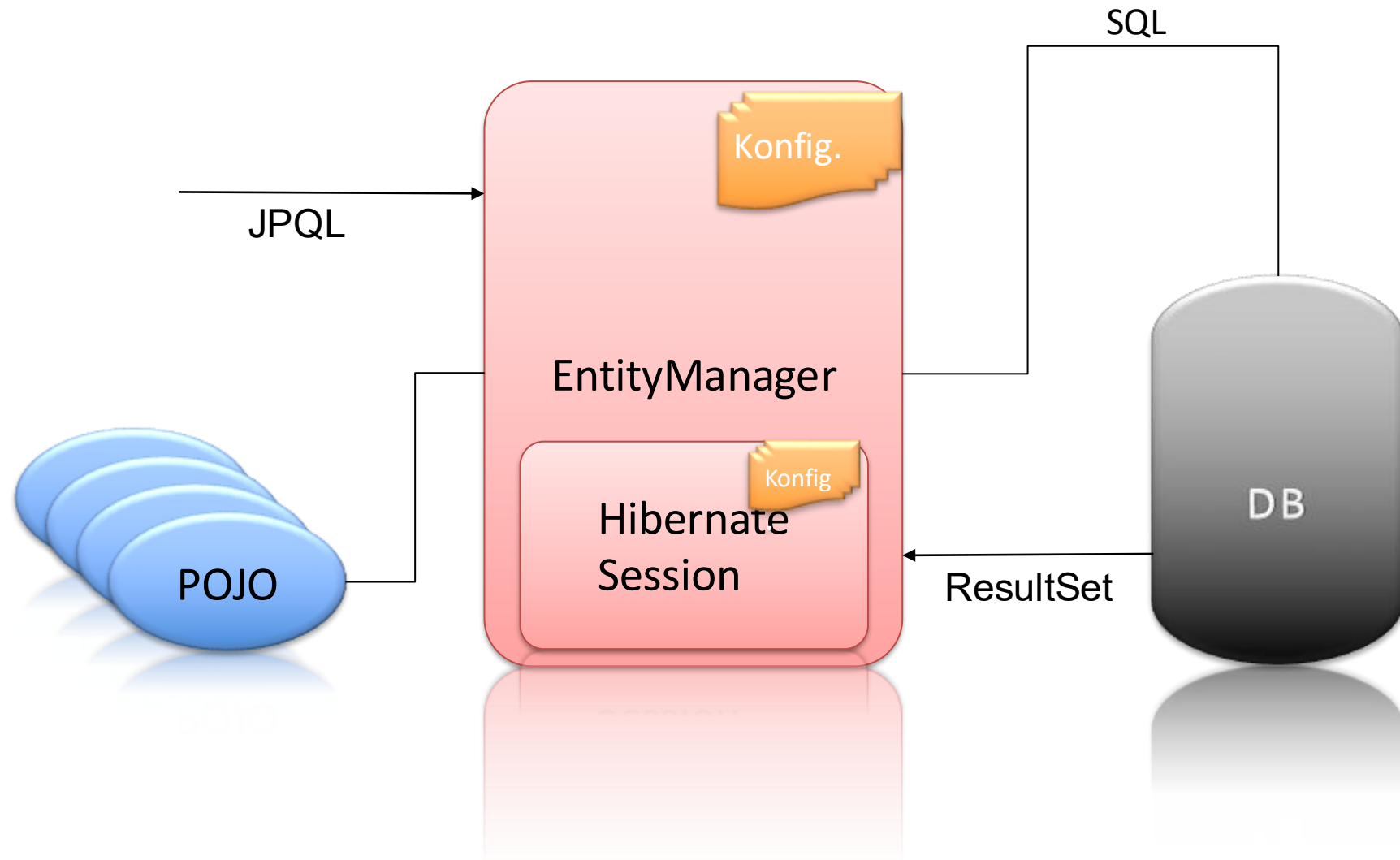
Zentrale Mapping Komponente

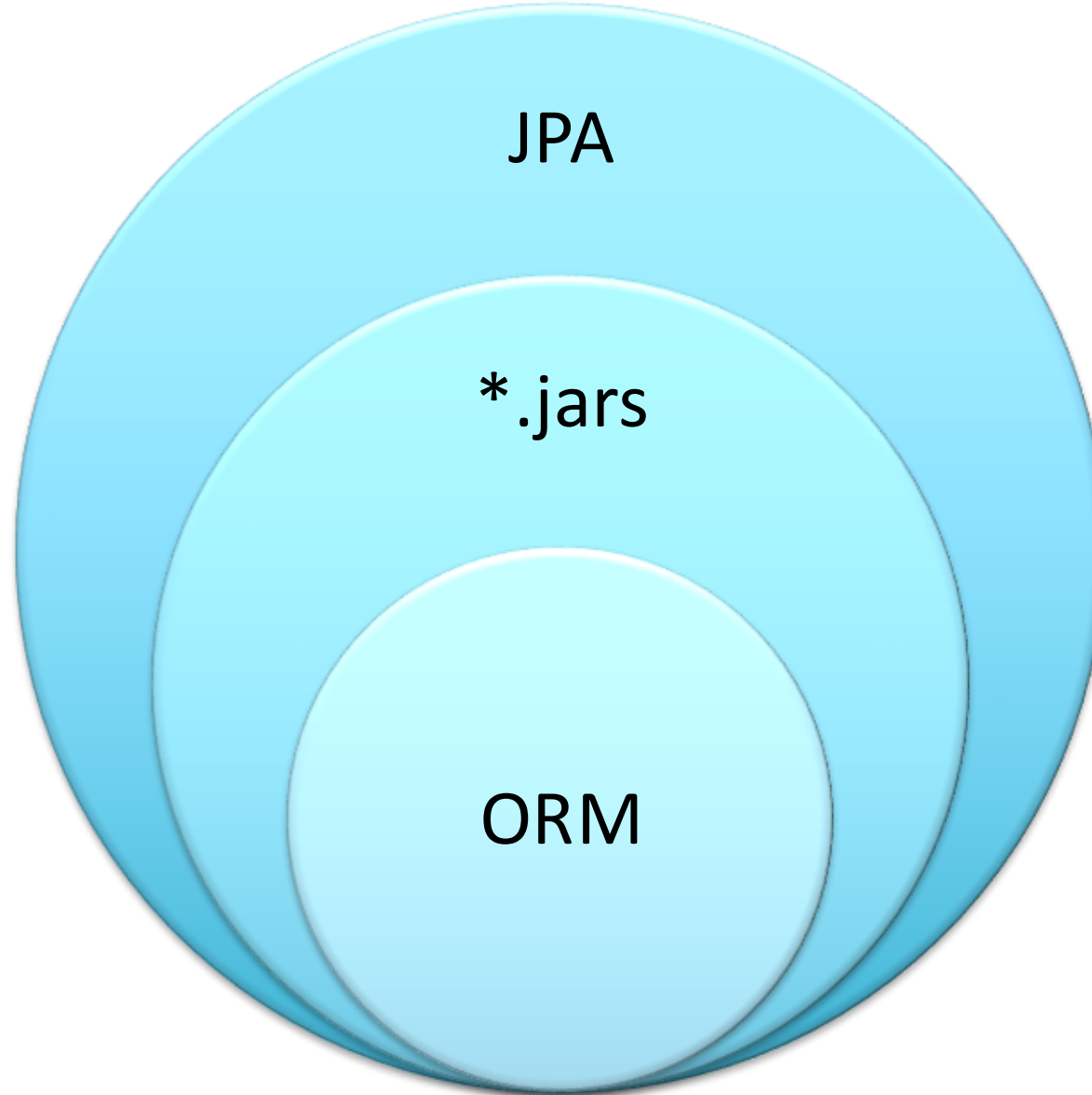


Hibernate Session

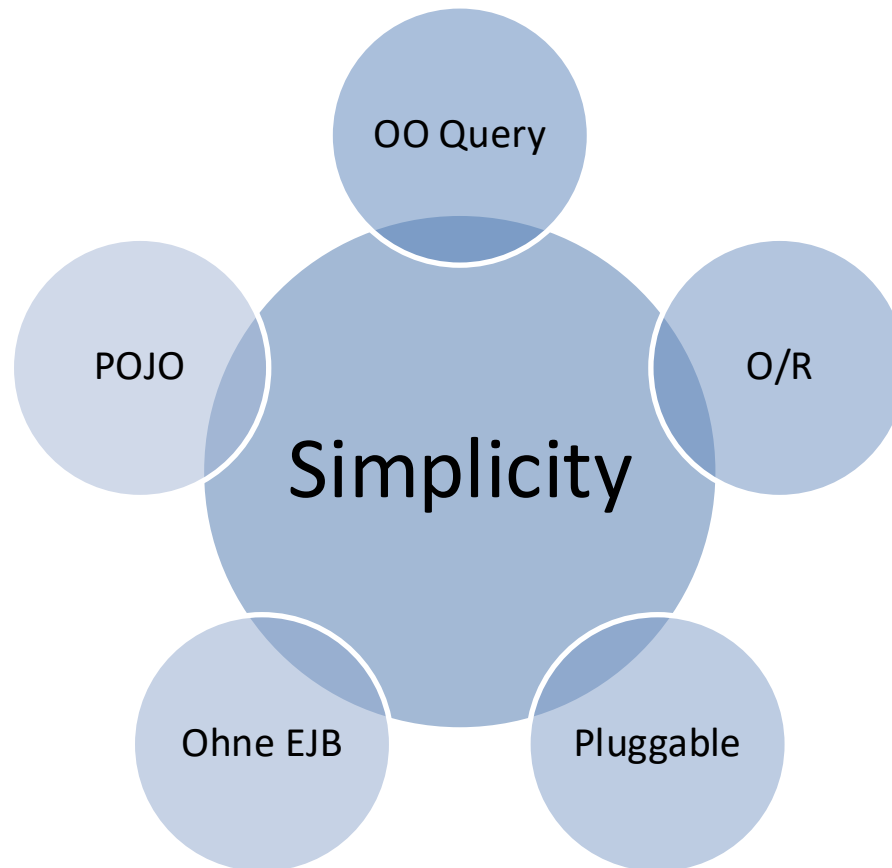


JPA





What developers **Need**



Ergebnis

Weniger Klassen und Interfaces

Keine längliche Deployment
Descriptors nötig

Configuration by Default

Kein Lookup Code

Unterstützt

- Vererbung
- Polymorphie und polymorphe Abfragen

Named Queries

Nutzbar außerhalb des EJB
Containers

- Testbarkeit

Pluggable Persistence Provider

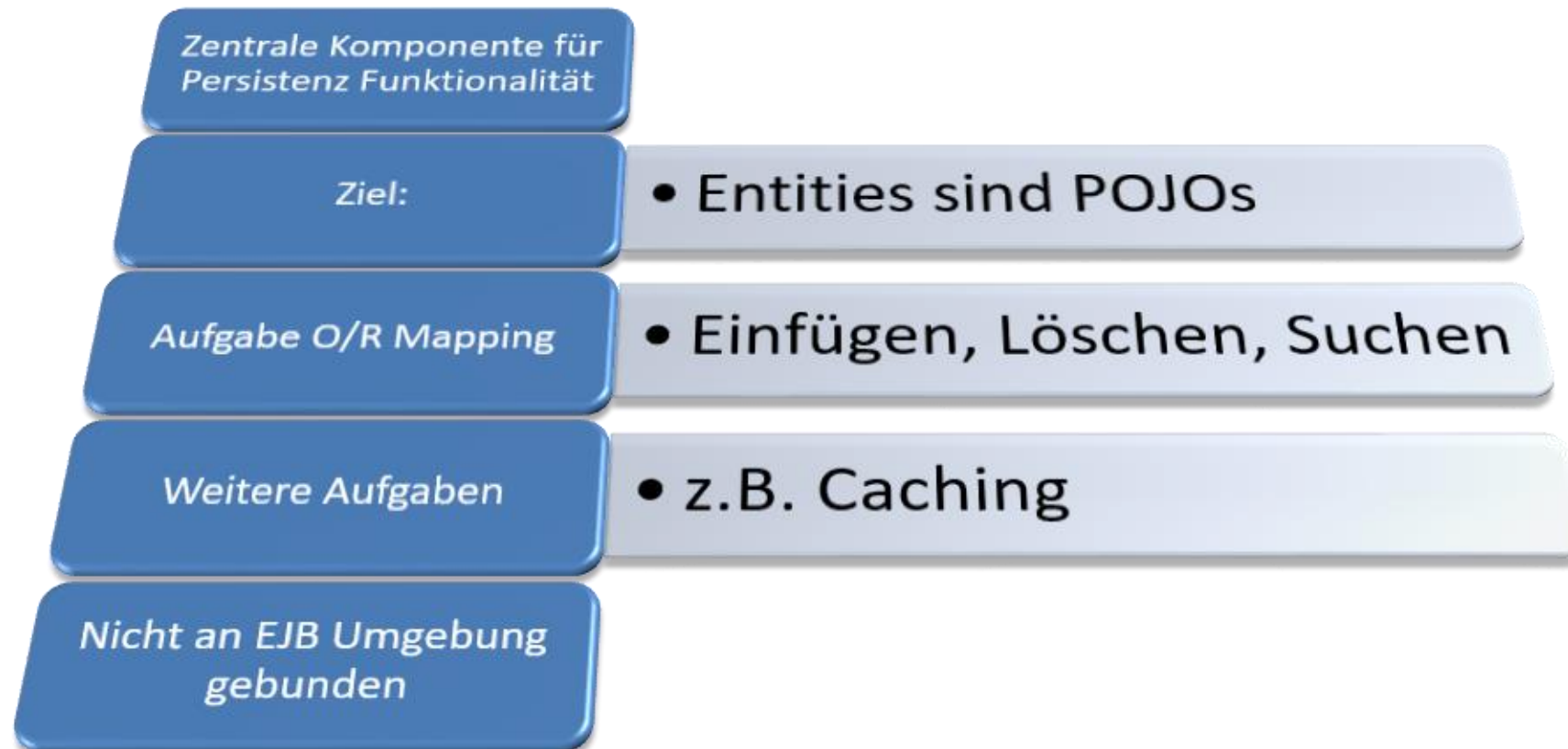
JPA 1.0

```
@Entity
```

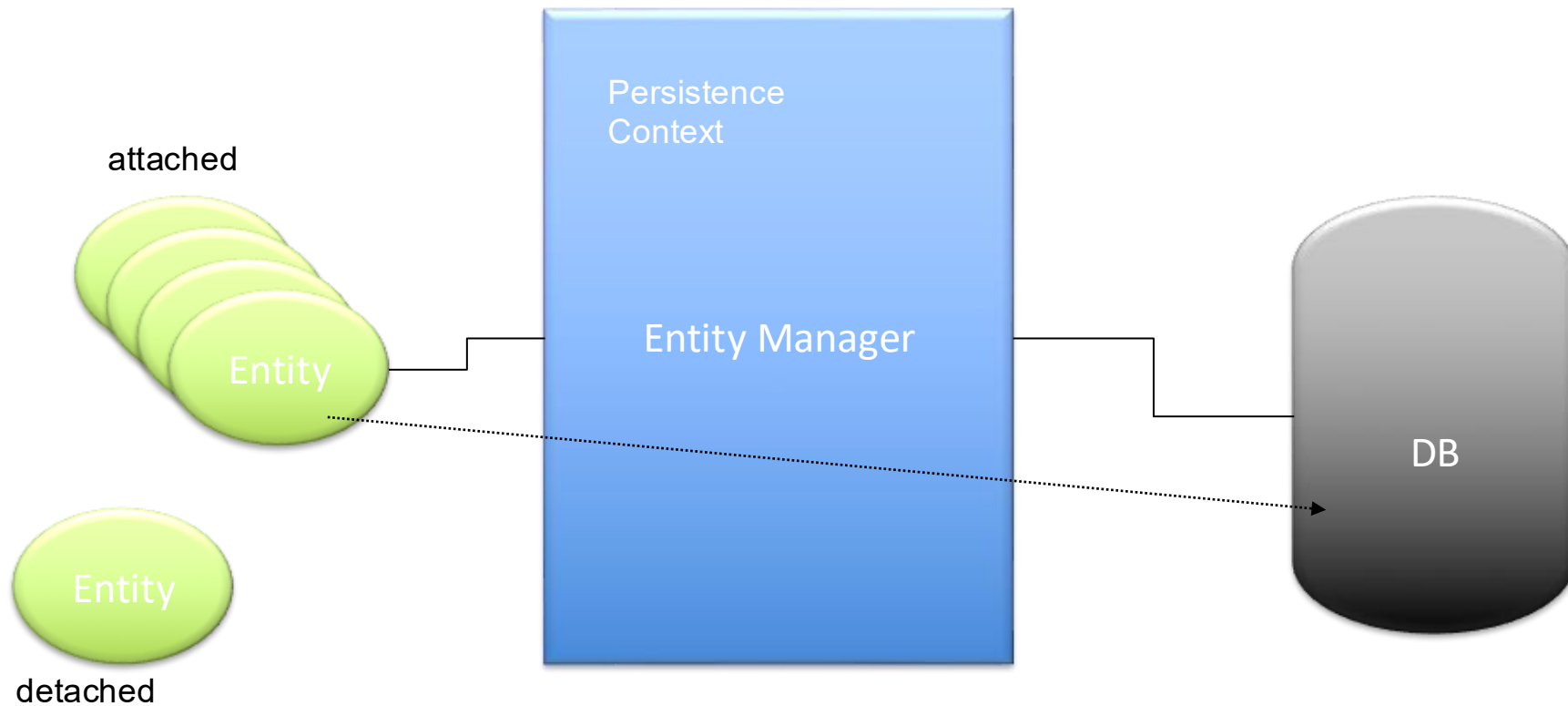
```
public class Adress {
```

```
}
```

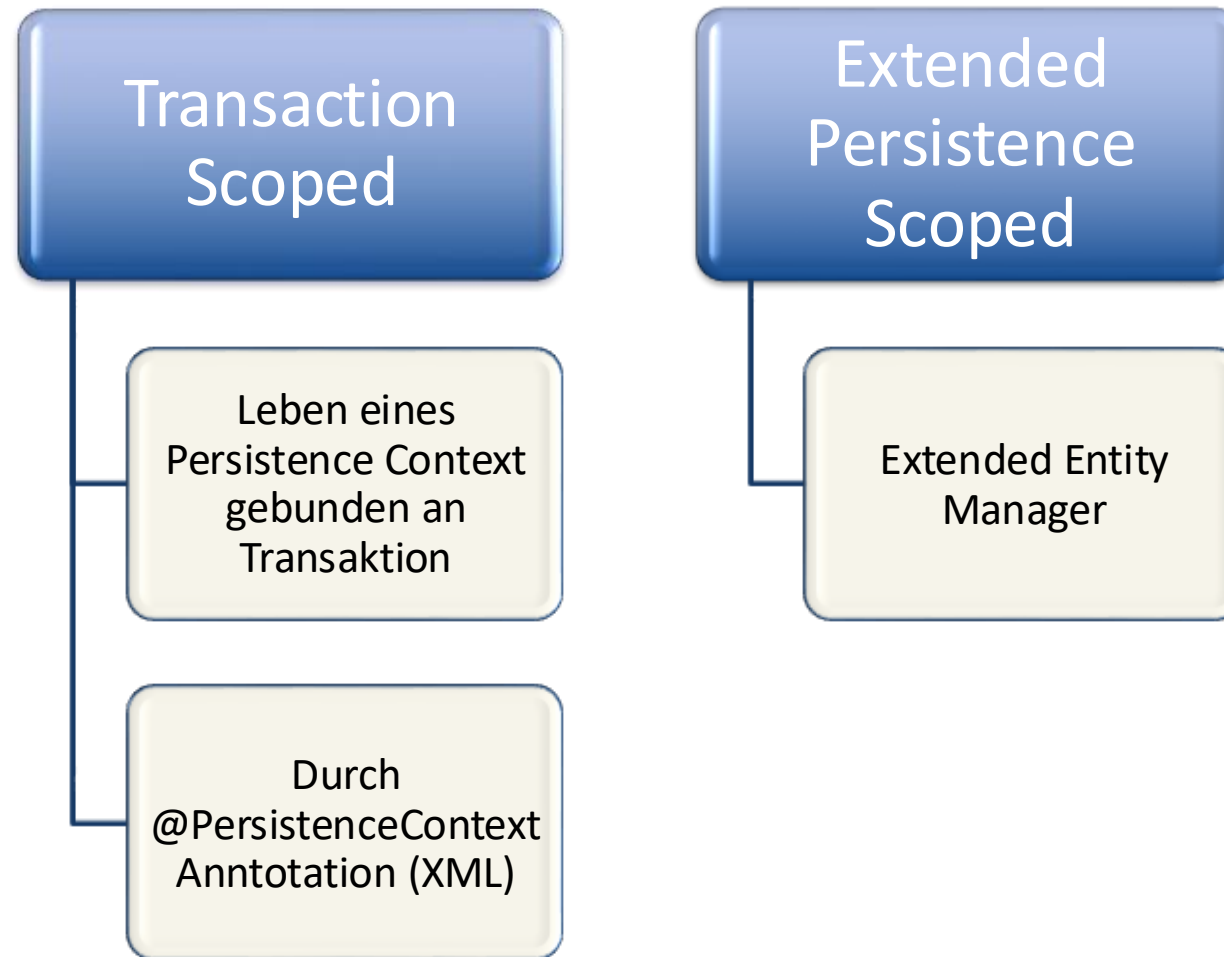
Grundlagen Entity Manager



Entity Lebenszyklus



Persistence Context



persistence.xml – so sieht sie in diesem Projekt aus

Modul 1_1_JPA_DB_CONFIG

- Jakarta Persistence 3.x, Namespace jakarta.ee – nicht mehr javax.
- transaction-type=RESOURCE_LOCAL: wir steuern die Transaktion selbst.
- Provider Hibernate, Datenbank Derby, Schema rentacar.

```
<persistence version="3.0"
  xmlns="https://jakarta.ee/xml/ns/persistence">
  <persistence-unit name="rentacar" transaction-type="RESOURCE_LOCAL">
    <provider>org.hibernate.jpa.HibernatePersistenceProvider</provider>
    <properties>
      <property name="jakarta.persistence.jdbc.url"
        value="jdbc:derby:rentacar;create=true"/>
      <property name="hibernate.hbm2ddl.auto" value="create-drop"/>
      <property name="hibernate.show_sql" value="true"/>
    </properties>
  </persistence-unit>
</persistence>
```

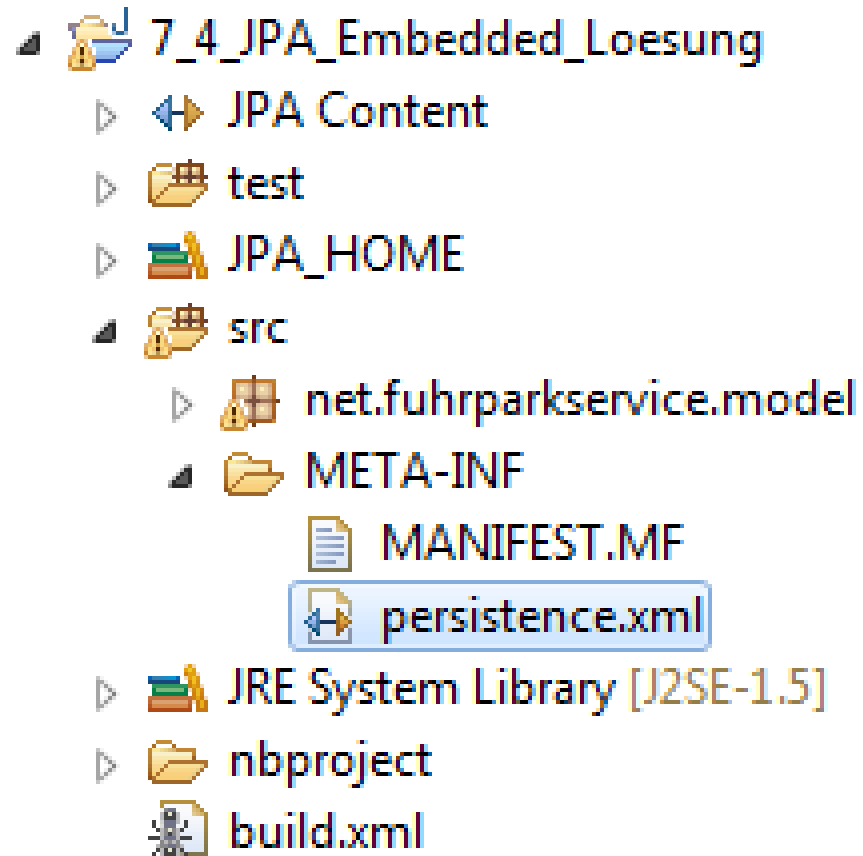
Optional: derselbe Code, anderer Provider

- JPA ist eine Spezifikation – Hibernate und EclipseLink sind Implementierungen.
- Im Projekt liegt dafür ein fertiges Maven-Profil bereit.
- Was sich ändert: generiertes SQL im Detail, Fehlermeldungen, Tuning-Optionen.
- Was gleich bleibt: Ihr Entity-Code.

```
mvn test                # Hibernate (Standard)
mvn -Peclipselink test  # EclipseLink – identischer Entity-Code
```

Persistence.xml

- META-INF Verzeichnis
 - JAR
 - EJB JAR
 - WAR (WEB-INF/lib)
 - EAR/lib
- Identifiziert und Beschreibt n Persistence Units
- Auflistung / Suchen(Annotation) der Entities



Persistence Unit

```
<persistence version="1.0"
  xmlns="http://java.sun.com/xml/ns/persistence" ..
  xsi:schemaLocation="...">
  <persistence-unit name="projectUnit" transaction-
    type="RESOURCE_LOCAL">
    <provider>org.eclipse.persistence.
      jpa.PersistenceProvider</provider>
    </properties>
  </persistence-unit>
</persistence>
```

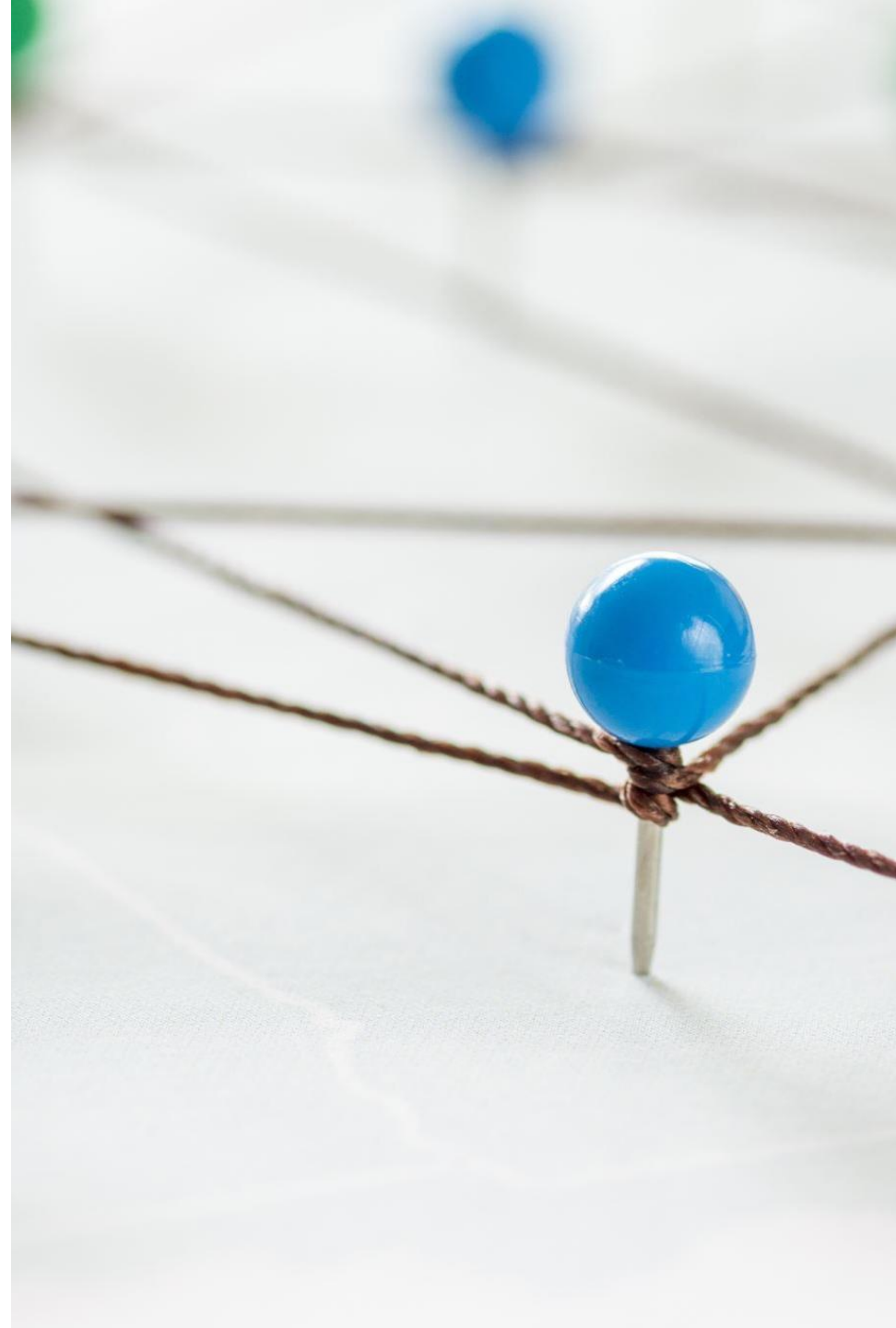
Entity Manager

- `persist(Object entity)`
- `find(Class <T> entityClass, Object key)`
- `getReference(Class <T> entityClass, Object key)`
 - throws `EntityNotFoundException`
- `merge(T entity)`
- `remove(Object entity)`
- `lock(Object entity, LockModeType lockMode)`
- `refresh(Object entity)`
- `contains(Object entity)`
- `clear()`

Entity Manager - 2

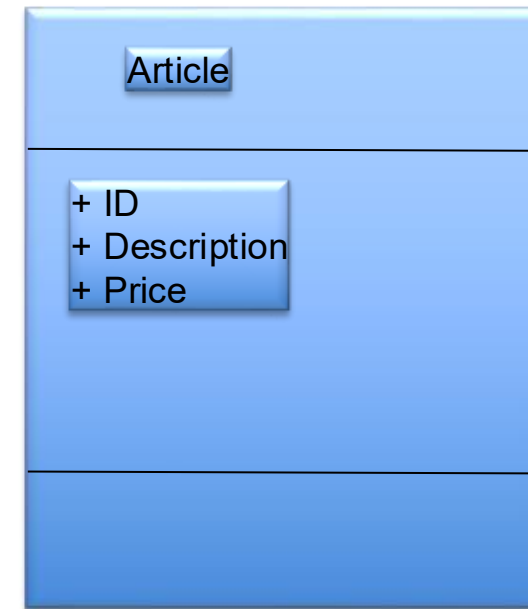
- `createQuery(String queryString)`
- `createNamedQuery(String name)`
- `createNativeQuery(String sqlString)`
- `createNativeQuery(String sqlString, String resultSetMapping)`
- `createNativeQuery(String sqlString, Class resultClass)`
- `getDelegate()`

- **Objekt Relationales Mapping**



OR Mapping

ID	Description	Price
1	Java ist auch eine Insel	49,99 €
2	Hibernate	39,99 €



Application



Database

<properties>

```
<property name="javax.persistence.jdbc.driver" value="org.apache.derby.jdbc.ClientDriver" />
<property name="eclipselink.target-database" value="Derby" />
<property name="javax.persistence.jdbc.url"
           value="jdbc:derby://localhost:1527/sun-appserv-samples;create=true" />
<property name="javax.persistence.jdbc.user" value="APP" />
<property name="javax.persistence.jdbc.password" value="a" />
```

<properties>

```
<property name="eclipselink.ddl-generation" value="drop-and-create-tables" />  
<property name="eclipselink.ddl-generation.output-mode"  
          value="database" />  
<property name="eclipselink.logging.level" value="FINEST" />  
<property name="eclipselink.logging.parameters" value="true"/>
```

<properties>

```
<property name="hibernate.hbm2ddl.auto" value="create" />  
<property name="hibernate.dialect"  
    value="org.hibernate.dialect.DerbyDialect" />
```

```
EntityManagerFactory managerFactory =  
    Persistence.createEntityManagerFactory("projectUnit");
```

```
EntityManager manager = managerFactory.createEntityManager();  
manager.isOpen();
```

persistence.xml

```
<persistence version="1.0"  
xmlns="http://java.sun.com/xml/ns/persistence" ..>  
    <persistence-unit name="projectUnit"
```

Aufgabe 1_1

- Maven Modul 1_1_JPA_DB_CONFIG zu finden
 - Konfiguration der Datei `src/META-INF/persistence.xml`.
 - Füge den `<provider>` für Hibernate hinzu
 - Konfiguriere die Datenbankverbindung mit den Infos aus der vorangegangenen Übung (url, user, password, driver class)
 - Hibernate spezifische Properties
 - `hibernate.dialect`:
`org.hibernate.dialect.DerbyDialect`
 - `hibernate.hbm2ddl.auto`: `create-drop`
 - `hibernate.show_sql`: `true`
(optional, zur Fehlersuche hilfreich)
 - Verbindung testen in `TestConnection`
- Der Test ist erfolgreich, wenn die `EntityManagerFactory` korrekt erstellt werden kann und `entityManager.isOpen` true returned



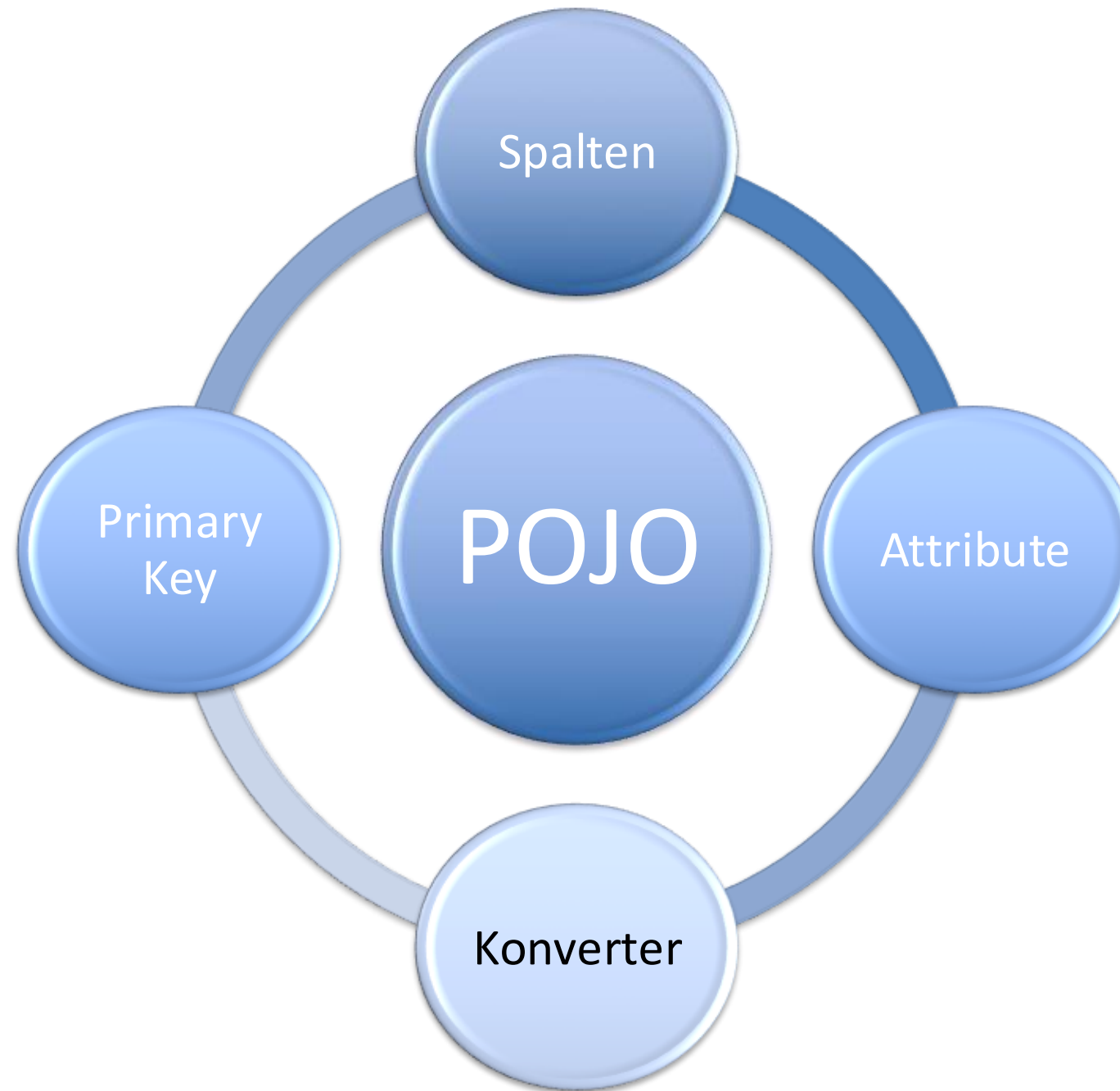
Transaktionen in unserem Setup

Was jede Übung im Hintergrund tut

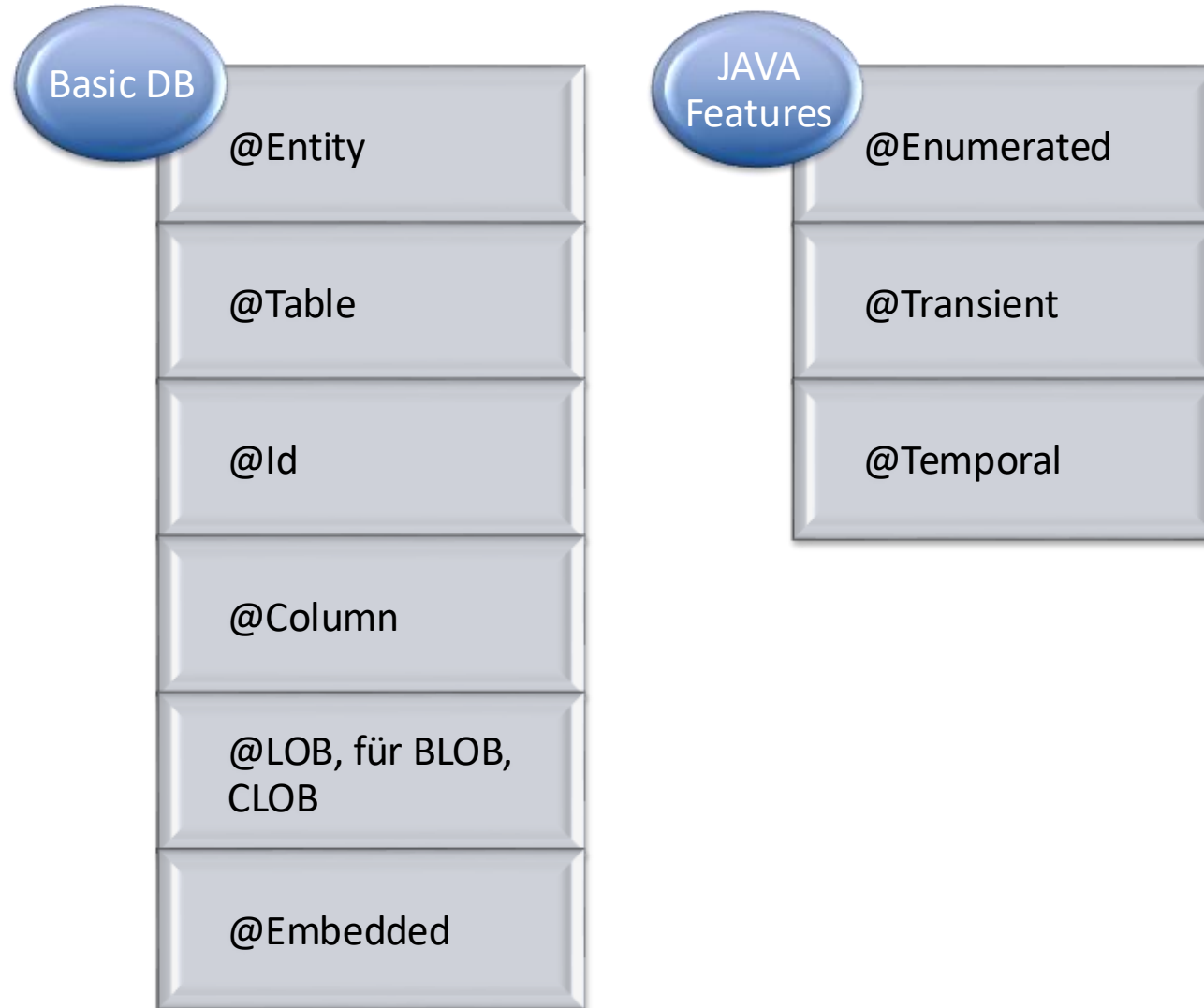
- Kein Container, kein JTA: wir arbeiten mit RESOURCE_LOCAL und EntityTransaction.
- Die Testbasisklasse öffnet pro Test einen EntityManager und rollt am Ende zurück.
- Deshalb sind alle Übungen voneinander unabhängig und beliebig wiederholbar.
- Mit dem Profil -Pinspect lesen Sie das erzeugte SQL mit – Basis für alle Predict-then-Run-Übungen.

```
em.getTransaction().begin();  
em.persist(new VehicleType("VW", "Golf"));  
em.getTransaction().commit(); // hier erst fällt das INSERT
```

```
mvn -Pinspect test          # SQL im Log mitlesen
```



Basic Mapping



@Entity

```
@Entity()
```

```
public class Article
```

- `@Entity(name="ArticleEntity")`
 - Angabe des Entitynamens möglich
 - Configuration by default → unqualifizierter Klassenname = Entityname
 - Wichtig für spätere Queries mit EJBQL Generierung

@Table

- Angabe der Tabelle in der das Entity gespeichert werden soll
 - Configuration by default: Tabellennamen = unqualifizierter Klassenname

```
@Table(name = "tbl_article", schema = "hr",  
        uniqueConstraints = {  
            @UniqueConstraint(columnNames = { "article_nr" })  
        })
```

Aufgabe 1_2_1

- Mappen Sie die ID der Klasse VehicleType als minimale Entity
- Testen Sie das Mapping mit der Testklasse
- Der folgende Unittest läuft grün

```
@Test
public void testFind() {
    //SELECT * from VehicleType where id= ...
    VehicleType VehicleType =
super.manager.find(VehicleType.class, id);
    assertNotNull(VehicleType);
    assertEquals(id, VehicleType.getId());
}
```

- Überprüfen Sie das erzeugte DB Schema



@Column

- Configuration per default: Spaltenname = Feldname
- Abweichend:
 - columnDefinition : String
 - name : String
 - table : String
 - insertable : boolean
 - length : int
 - precision : int
 - scale : int
 - nullable : boolean
 - unique : boolean
 - updateable : boolean

@Id

```
private String id;
```

Primary Key



ID-Strategien – Entscheidungsmatrix

Module 1_2_2 bis 1_2_4

Strategie	Wie	JDBC-Batch	Typisch für	Achtung
IDENTITY	Auto-Increment der DB	nein	MySQL, SQL Server	Insert kann nicht verzögert werden
SEQUENCE	DB-Sequenz, allocationSize	ja	Oracle, PostgreSQL, Derby	Sequenz muss existieren
TABLE	eigene Schlüsseltabelle	ja	portabel, Legacy	Contention, extra Zugriff
UUID	UUID, seit JPA 3.1	ja	verteilte Systeme, Offline	größere Indizes
AUTO	Provider entscheidet	je nachdem	Prototypen	Verhalten je Provider verschieden

Faustregel: SEQUENCE mit allocationSize ist der Default unter Hibernate – IDENTITY verhindert das Batching von Inserts.

Example Simple IdGenerator

- `@Id`
`@GeneratedValue(strategy=GenerationType.AUTO)`
`private Long id;`

Example Sequence Generator

- `@SequenceGenerator(name = "Example_GEN", sequenceName = "DB_Example_SEQ")`
`@Id`
`@GeneratedValue(strategy = GenerationType.SEQUENCE,`
`generator = "Example_GEN")`
`private Long id;`

Example Table Generator

- `@TableGenerator(name = "Example_GEN", table = "tbl_ExampleGen", pkColumnName = "PK_COLUMN", valueColumnName = "VALUE_COLUMN", pkColumnValue = "Example_ID", allocationSize = 10)`
`@Id`
`@GeneratedValue(strategy = GenerationType.TABLE, generator = "Example_GEN")`
`private Long id;`

tbl_ExampleGen

PK_COLUMN	VALUE_COLUMN
EXAMPLE_ID	12
USER_ID	12



Performance:

IDENTITY bei Batch Inserts eher schlecht.



Database Compatibility:

Nicht alle Datenbanken unterstützen alle Strategien.



Customization:

Sequence Name, InitialValue, oder Increment können über Annotations angepasst werden.

Übung: ID-Strategien ausprobieren

🕒 15 min

Level ●●○

Modul 1_2_2 / 1_2_3 / 1_2_4

AUFTRAG

- 1_2_2: GenerationType.AUTO stellen und im SQL-Log prüfen, wofür Hibernate sich auf Derby entscheidet.
- 1_2_3: auf @TableGenerator umstellen – Tabellenname, pkColumnValue und allocationSize bewusst setzen.
- 1_2_4: auf @SequenceGenerator umstellen, allocationSize = 50.
- Vor dem Lauf notieren: Wie viele SELECTs auf die Sequenz erwarten Sie bei 100 Inserts?

ERFOLGSKRITERIEN

- ✓ Test läuft grün.
- ✓ Im Log ist die erzeugte Tabelle bzw. Sequenz sichtbar.
- ✓ Sie können begründen, warum allocationSize die Anzahl der Sequenzzugriffe senkt.

```
@Id
@GeneratedValue(strategy = GenerationType.SEQUENCE, generator = "vt_seq")
@SequenceGenerator(name = "vt_seq", sequenceName = "VT_SEQ", allocationSize = 50)
private long id;
```

Methode Predict-then-Run: erwartetes SQL vor dem Testlauf aufschreiben, danach mit dem Log vergleichen.

Composite Keys – Motivation & Einsatz

- In relationalen Datenbanken bestehen Primärschlüssel häufig aus **mehreren Spalten**, z. B.:
 - Bestellnummer + Positionsnummer
 - Benutzer-ID + Rollenname
 - Fahrzeug-ID + Standort + Datum
- In JPA müssen solche **zusammengesetzten Schlüssel explizit modelliert** werden. Dafür gibt es **zwei offizielle Strategien**:
 1. **@EmbeddedId** (empfohlen)
 2. **@IdClass** (Legacy / Sonderfälle)

Composite Keys in JPA – Überblick

Strategie	Kurzbeschreibung	Strategie
@EmbeddedId	Schlüssel als eigenes Value Object	@EmbeddedId
@IdClass	Schlüsselattribute direkt in der Entity	@IdClass

 **Best Practice:**

@EmbeddedId ist der empfohlene Standard

@EmbeddedId – Der Schlüssel als Value Object

- Der Primärschlüssel wird als **eigene Klasse modelliert**
- Diese Klasse ist **kein Entity**
- Sie wird vollständig in die Entity eingebettet
- Entspricht dem **OO-Value-Object-Gedanken**

@Embeddable

```
public class OrderItemId implements Serializable {  
  
    private Long orderId;  
    private Integer position;  
  
    public OrderItemId() {}  
  
    public OrderItemId(Long orderId, Integer position) {  
        this.orderId = orderId;  
        this.position = position;  
    }  
  
    @Override  
    public boolean equals(Object o) {  
        ...  
    }  
  
    @Override  
    public int hashCode() {  
        ...  
    }  
}
```

@EmbeddedId: Entity Mapping

- Kein zusätzliches Mapping nötig
- JPA erzeugt automatisch einen **Composite Primary Key**

```
@Entity
@Table(name = "ORDER_ITEM")
public class OrderItem {
```

```
    @EmbeddedId
    private OrderItemId id;
```

- SELECT oi
- FROM OrderItem oi
- WHERE oi.id.orderId = :orderId

```
    private String articleName;
    private int quantity;
}
```

@IdClass – Alternative Strategie

- Schlüsselattribute liegen **direkt in der Entity**
- Key-Klasse enthält nur **Duplikate**
- Schwächere Kapselung
- Höhere Fehleranfälligkeit
-  **Nur für Sonderfälle oder Legacy Code**

@IdClass – Beispiel

```
public class OrderItemId  
implements Serializable {  
  
    private Long orderId;  
  
    private Integer position;  
  
}
```

```
@Entity  
@IdClass(OrderItemId.class)  
public class OrderItem {
```

```
    @Id  
    private Long orderId;
```

```
    @Id  
    private Integer position;
```

```
    private String article
```

@EmbeddedId vs. @IdClass

Kriterium	EmbeddedId	IdClass
OO-Design	✅ gut	❌ schlecht
Wartbarkeit	✅ hoch	❌ niedrig
JPQL	✅ sauber	⚠️ unübersichtlich
Empfehlung	✅ Standard	❌ vermeiden

Wenn ihr euch entscheiden müsst:

👉 **EmbeddedId ist fast immer die richtige Wahl**

Typische Fehler bei Composite Keys

-  equals/hashCode vergessen
-  mutable Key-Klassen
-  @GeneratedValue bei Composite Keys
-  @IdClass ohne Not
-  Composite Keys sind **fachlich stabil**

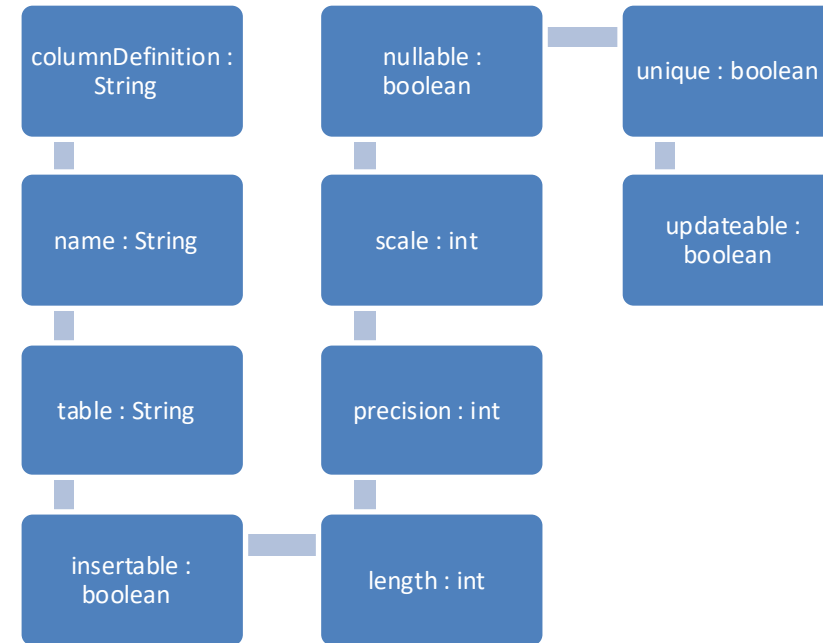
Zusammengesetzte Schlüssel: @IdClass oder @EmbeddedId?

Module 1_2_5 und 1_2_6

	@IdClass	@EmbeddedId
Entity	einzelne Felder mit @Id	ein Feld vom Typ der ID-Klasse
ID-Klasse	separate Klasse, gleiche Feldnamen	@Embeddable, wird Teil des Entities
JPQL	v.kennzeichen	v.id.kennzeichen
Lesbarkeit	näher am klassischen SQL-Denken	die ID ist ein eigenes Objekt mit Bedeutung
Empfehlung	Legacy-Schemata	Neuentwicklung

Beide brauchen: Serializable, equals und hashCode, öffentlicher Standardkonstruktor.

Attribute <--
> Column



@Column

```
@Column(name="DESC", nullable=false, length=512)  
public String getDescription() { return description; }
```

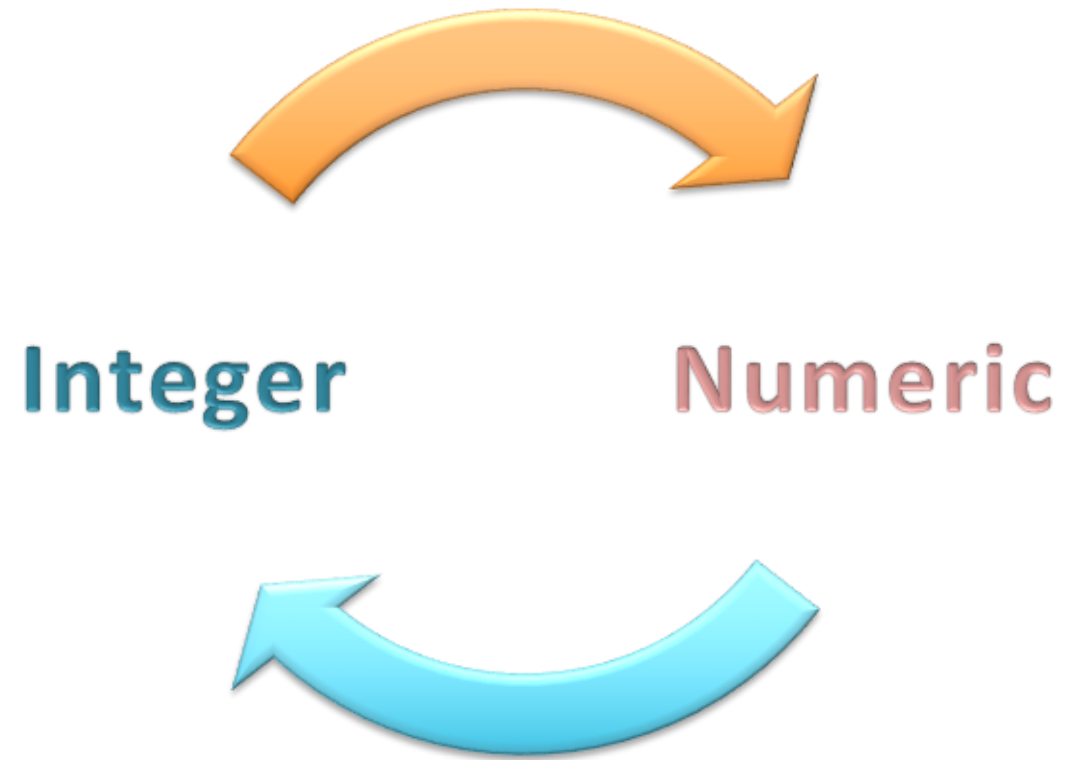
```
@Column(name="DESC", columnDefinition="CLOB NOT NULL",  
table="EMP_DETAIL")
```

```
@Lob
```

```
public String getDescription() { return description; }
```

```
@Column(name="ORDER_COST", updatable=false, precision=12,  
scale=2)
```

```
public BigDecimal getCost() { return cost; }
```



Typeconverter **by DEFAULT**

Aufgabe 1_3

- Beschreiben Sie das Mapping der Attribute der Klasse FahrzeugType wie folgt:
 - brand länge=30 Pflichtfeld
 - modell länge = 50 Pflichtfeld
 - hp vierstelliger Integer
 - maxKpH dreistelliger Integer
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





Kleines Quiz

1. Wie wird die maximale Spaltenlänge angepasst?
 - Und was passiert wenn das Attribut mit einem längeren Wert befüllt wird.
2. Welcher Datentyp braucht man javaseitig um Precision (NUMBER Datentyp) wirksam werden zu lassen.

AttributeConverter – eigene Typen abbilden

Modul 1_3_1

- Für Werte, die JPA nicht kennt, oder für eine kompakte Datenbankdarstellung.
- `autoApply = true` gilt für alle Felder dieses Typs, sonst gezielt mit `@Convert`.
- Abgrenzung: `@Enumerated(String)` speichert den Enum-Namen – der Converter erlaubt eine eigene, stabile Codierung.
- Achtung: in JPQL wird auf der konvertierten Spalte verglichen.

```
@Converter(autoApply = true)
public class StatusConverter implements AttributeConverter<Status, String> {

    public String convertToDatabaseColumn(Status s) {
        return s == null ? null : s.getCode();    // VERFUEGBAR -> "V"
    }

    public Status convertToEntityAttribute(String c) {
        return Status.of(c);
    }
}
```

Übung: Fahrzeugstatus als Code speichern

🕒 15 min

Level ●●○

Modul 1_3_1

AUFTRAG

- Enum Status (VERFUEGBAR, VERMIETET, WARTUNG) an Vehicle ergänzen.
- AttributeConverter schreiben, der den Status als ein Zeichen persistiert.
- Converter mit autoApply = true registrieren.
- Im SQL-Log prüfen, was tatsächlich in der Spalte landet.

ERFOLGSKRITERIEN

- ✓ Test grün.
- ✓ In der Datenbank steht V, M oder W – nicht der Enum-Name.
- ✓ Sie können erklären, wann der Converter nicht greift.

Zusatz für Schnelle: Was passiert bei einer Native Query auf dieselbe Spalte? Warum?

Quiz: Wann wird der AttributeConverter aufgerufen?

- A) Nur beim Schreiben, das Lesen geht direkt über den Enum-Namen.
- B) Beim Schreiben und Lesen – auch bei JPQL-Parametern.
- C) Nur bei em.persist, nicht bei em.merge.
- D) Auch bei Native Queries, weil Hibernate das SQL analysiert.

Think – Pair – Share: 1 Minute allein, 1 Minute zu zweit, dann Auflösung.

Auflösung: AttributeConverter

B – beim Schreiben und Lesen, inklusive JPQL-Parameterbindung.

- `convertToDatabaseColumn` läuft beim Flush, `convertToEntityAttribute` beim Laden.
- Auch ein über `setParameter` gebundener Enum-Wert wird konvertiert.
- Native Queries gehen am Converter vorbei – dort steht der rohe Spaltenwert.
- Deshalb: Codes stabil halten, sie sind Teil Ihres Datenbankvertrags.

Recap Kapitel 1

Dieselben Lernziele – jetzt als Häkchenliste.

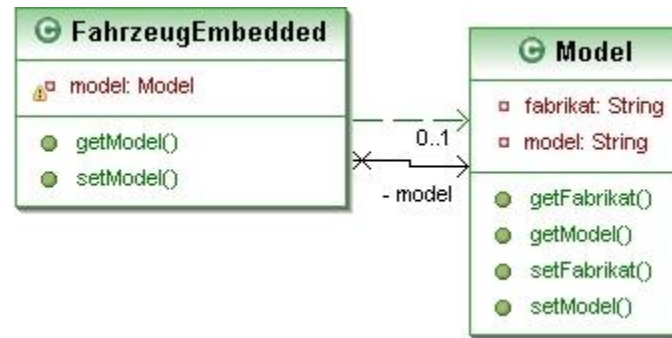
- ✓ Sie können erklären, was der Persistence Context leistet und was er kostet.
- ✓ Sie konfigurieren eine Persistence Unit für Hibernate mit `RESOURCE_LOCAL`.
- ✓ Sie wählen eine ID-Strategie begründet – inklusive Batching-Folgen.
- ✓ Sie bilden eigene Typen mit `AttributeConverter` und Wertobjekte mit `@Embedded` ab.

Als Nächstes: Beziehungen zwischen Entities – und was Kaskadierung wirklich auslöst.

OR Mapper Challenge

- Datentypen (Relational, OO)
- Identität – Primärschlüssel
- Mengen --> Spalten
- Kardinalität (1:n, n:m)
- Vererbung

@Embedded und @Embeddable



@Embeddable

```
@Embeddable  
public class Model {  
  
    private String model;  
    private String fabrikat;
```

@Embedded

```
@Entity
```

```
public class FahrzeugEmbedded {
```

```
    @Embedded
```

```
    @AttributeOverrides( {
```

```
        @AttributeOverride(name = "model", column =  
        @Column(name = "model_model")),
```

```
        @AttributeOverride(name = "fabrikat",  
        column = @Column(name = "model_fabrikat"))  
    })
```

```
private Model model;
```

Aufgabe 1_4

- Lagern Sie die Attribute fabrikat und modell der Klasse VehicleType in einer neuen Klasse Model ab.
- In der Datenbank sollen die beiden Attribute jedoch in der Tabelle tbl_VehicleType verbleiben
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





Kleines Quiz

1. Mehrstufig? Ist ein Embeddable in einem Embedded möglich?
 - Versuchen Sie es, z.B. ein `VehicleModel` als `Embedded`, welches Model enthält und z.B: noch ein Attribut *series*.
2. Wie kann ich den Spaltennamen eines embedded Attributs anpassen

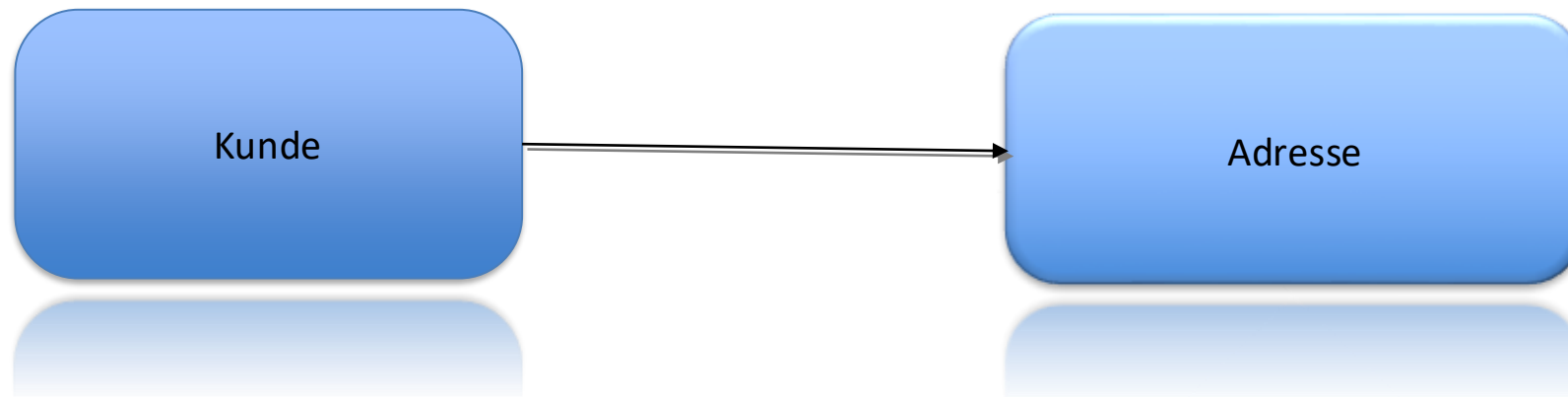
02

Assoziationen & Vererbung

Nach diesem Kapitel können Sie ...

- die vier Assoziationstypen korrekt und ohne Doppelspalten abbilden,
- cascade und orphanRemoval sicher unterscheiden,
- die drei Vererbungsstrategien am erzeugten SQL erkennen,
- @MappedSuperclass gegen echte Entity-Vererbung abgrenzen.

Assoziation



ARTEN

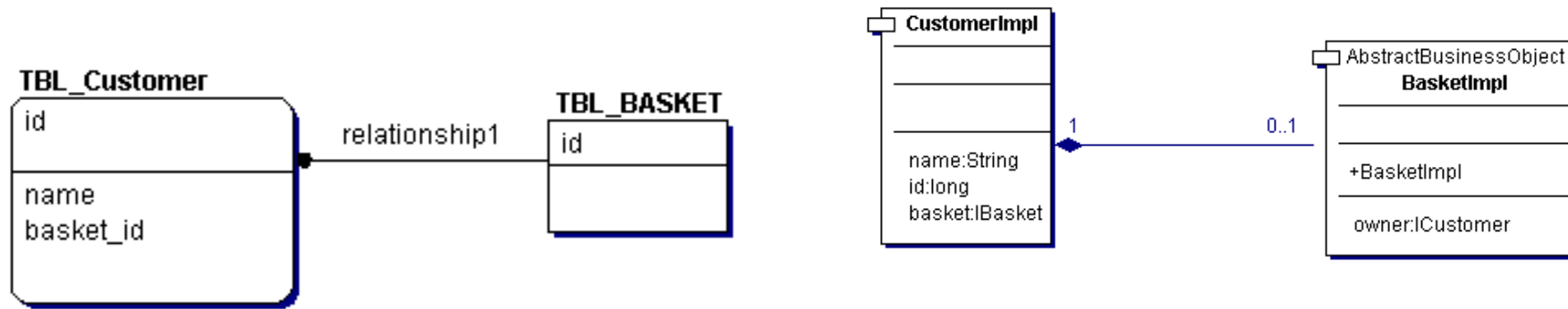


Die vier Assoziationen im Überblick

Module 1_5 bis 1_8

Annotation	Beispiel im Modell	Besitzende Seite	Fremdschlüssel
@OneToOne	User – Person	die Seite ohne mappedBy	Spalte in USER
@ManyToOne	Vehicle → VehicleType	immer die Many-Seite	Spalte in VEHICLE
@OneToMany	Shop → Vehicle (mappedBy)	die Many-Seite	Spalte in VEHICLE
@ManyToMany	Vehicle – Shop (Historie)	frei wählbar	eigene Zwischentabelle

mappedBy zeigt immer auf das Feld der Gegenseite – fehlt es, entstehen zwei Fremdschlüssel für dieselbe Beziehung.



One-To-One

```
@Entity
```

```
public class Customer {
```

```
...
```

```
@OneToOne
```

```
@JoinColumn (name="basket_id")
```

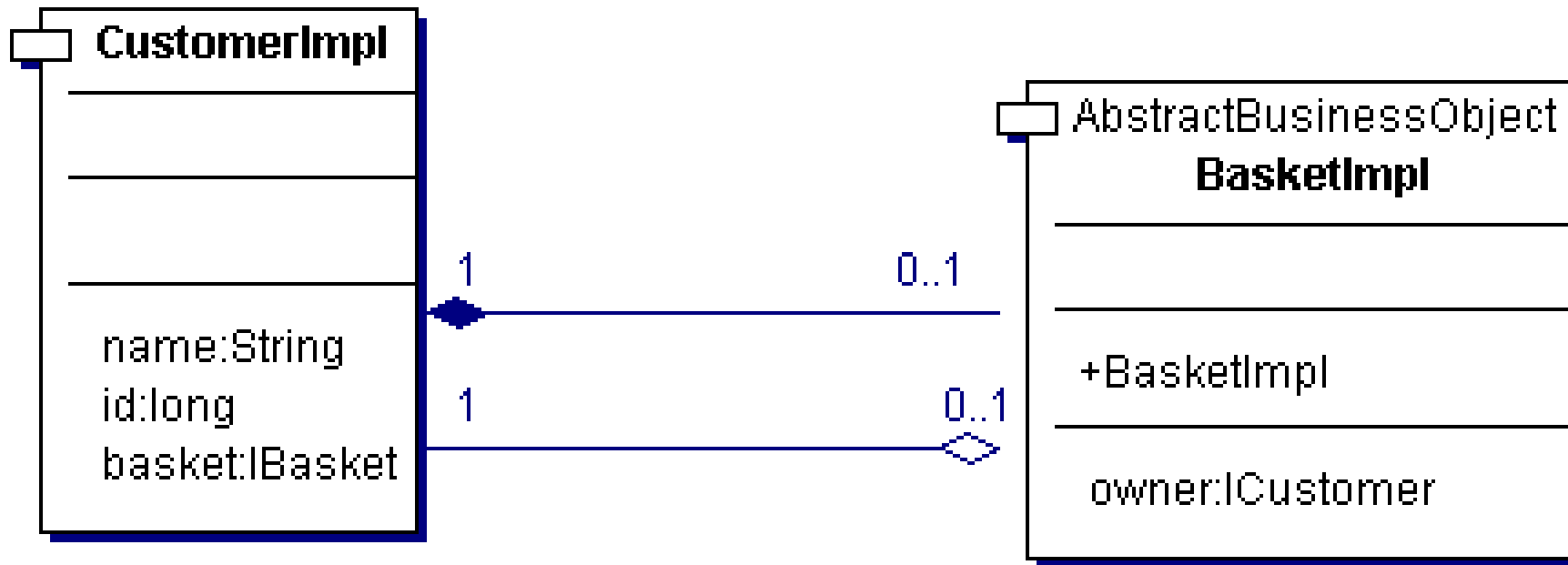
```
public Basket basket;
```

Implicit JoinColumn

@OneToOne

@PrimaryKeyJoinColumn

public Basket basket



One-To-One -
Bidirectional

```
public class Basket extends AbstractBusinessObject
implements IBasket {

    @OneToOne(mappedBy="basket")
    private Customer owner;
```

Aufgabe 1_5

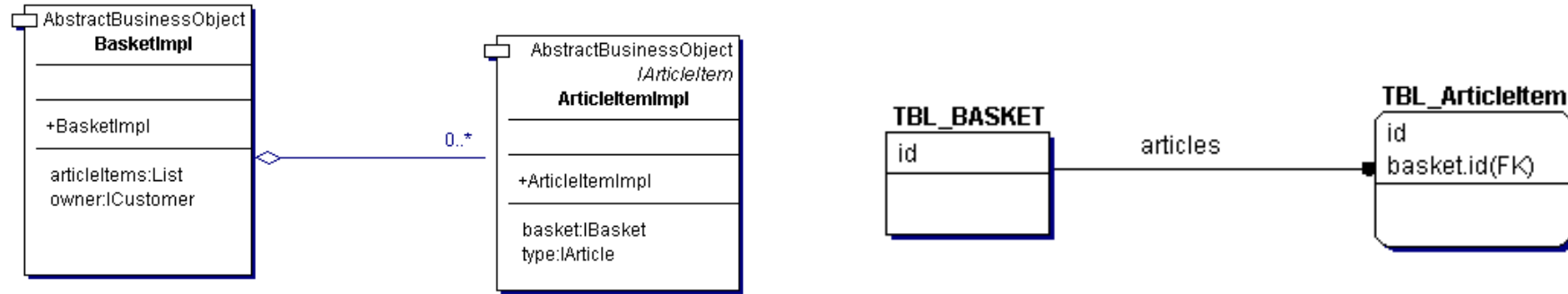
- Mappen Sie die Beziehung User -> Person
 - Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





Kleines Quiz

- In welcher Tabelle befindet sich die Fremdschlüsselspalte?



One-To-Many

```
@OneToMany()  
@JoinColumn(name="basket_id")  
private List<ArticleItem> getArticleItems();
```

```
@JoinTable(name="basket_article"  
    ,joinColumns={@JoinColumn(name="basket_id")}  
    ,inverseJoinColumns={@JoinColumn(name="articleItem_id")})
```

orphanRemoval=true



Constantin Film

AB 14. SEPTEMBER 2006 NUR IM KINO

CONSTANTIN FILM UND BERND EICHINGER ZEIGEN

DAS PARFUM

DIE GESCHICHTE EINES MÖRDERS

NACH DEM ROMAN VON PATRICK SÜSKIND
EIN TOM TYKWER FILM EINE BERND EICHINGER PRODUKTION

Aufgabe 1_6

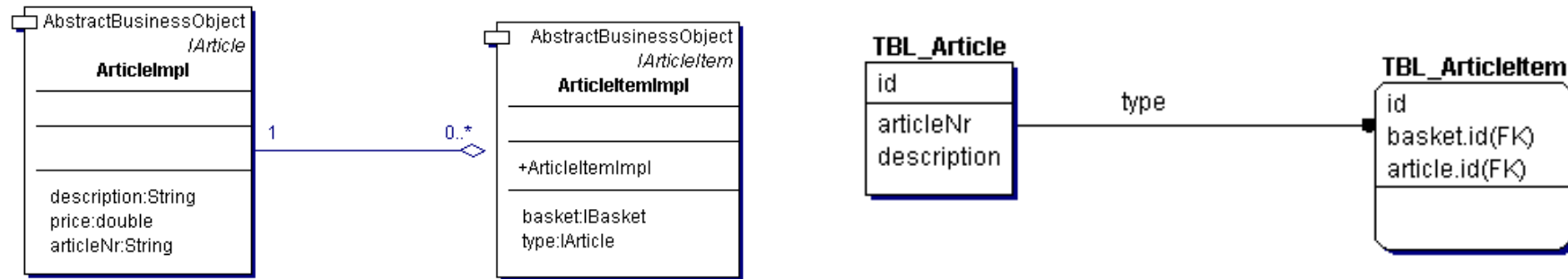
- Mappen Sie die Beziehung Shop -> Vehicle
- Mappen Sie die Vehicle Klasse
 - Die Attribute type und location sollen vom Persistenzprovider ignoriert werden
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





Kleines Quiz

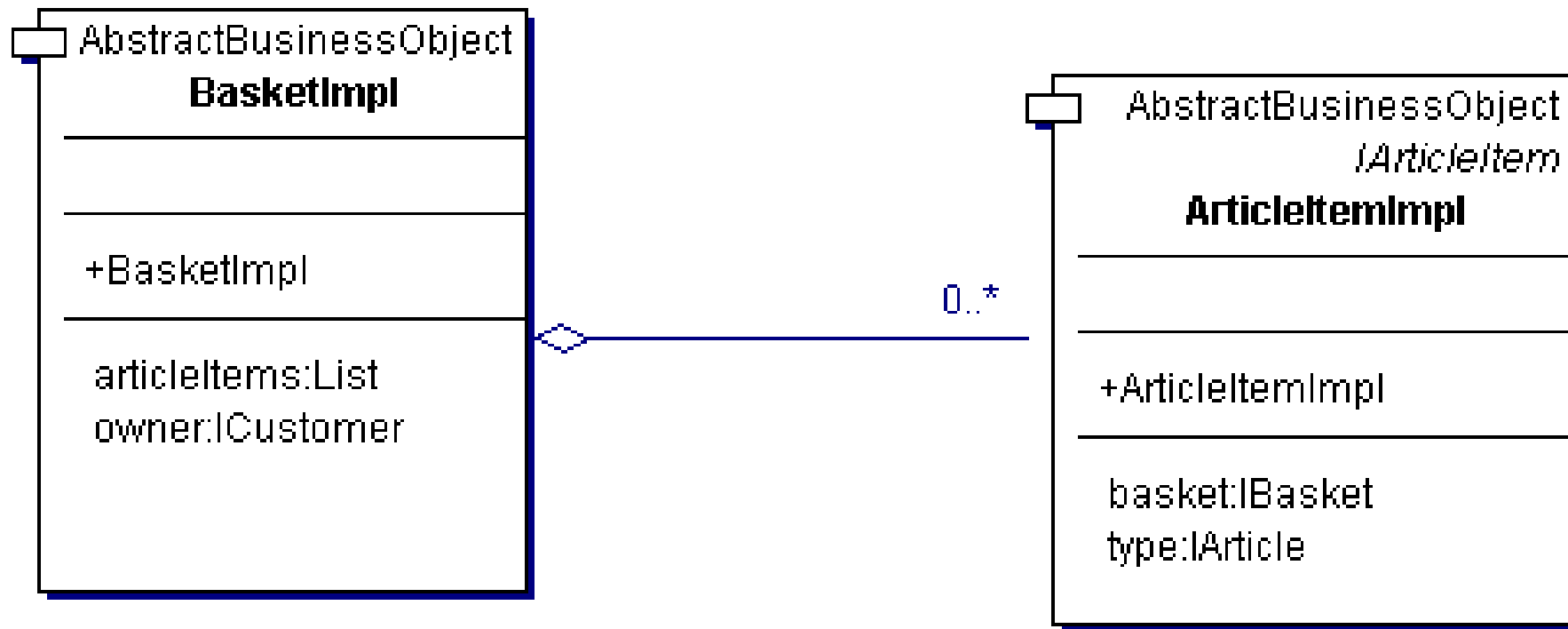
1. Welche Datentypen machen für eine 1:n Beziehung in Java nur einen Sinn?
2. Warum ist "vehicleId" kein guter Namen für die Fremdschlüsselbeziehung der in der Übung abgebildeten Beziehung



Many-To-One

Many-To-One

```
@ManyToOne  
@JoinColumn(name="article_id")  
Private Article type;
```



One-To-Many - Bidirectional

```
@OneToMany(mappedBy="basket")  
private List<ArticleItem> getArticleItems();
```

Aufgabe 1_7

- Mappen Sie die folgenden Attribute der Klasse Vehicle
 - location (Achtung bidirektional)
 - type (Viele Fahrzeuge haben den gleichen Typ)
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema





Kleines Quiz

1. Was müssten wir in `VehicleType` ergänzen, um die Rückrichtung der Beziehung zu modellieren.

<i>Collection Typ</i>	<i>Annotationen</i>	<i>Inhalt</i>	<i>Beschreibung</i>
java.util.List	@OrderBy	(<i>Spalte</i> ASC/DESC)	Reihenfolge auch in DB
java.util.Map	@MapKey	(name=“ <i>Spalte</i> “)	Keine definierte Reihenfolge hashCode und equals für Schlüssel und Werte Comparator

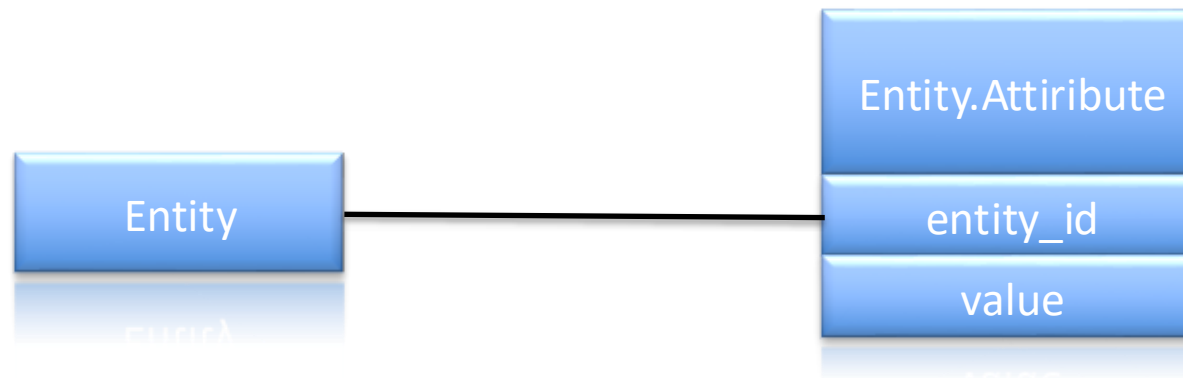
Collection Typen

A black and white photograph showing a perspective view of a garment factory. Numerous workers, mostly women, are seated at long tables equipped with sewing machines. They are all focused on their work. The factory has a high ceiling and industrial lighting. The workers are wearing light-colored, short-sleeved uniforms. The sewing machines are vintage-style treadle machines. The overall scene depicts a busy manufacturing environment from the mid-20th century.

Noch mehr List..

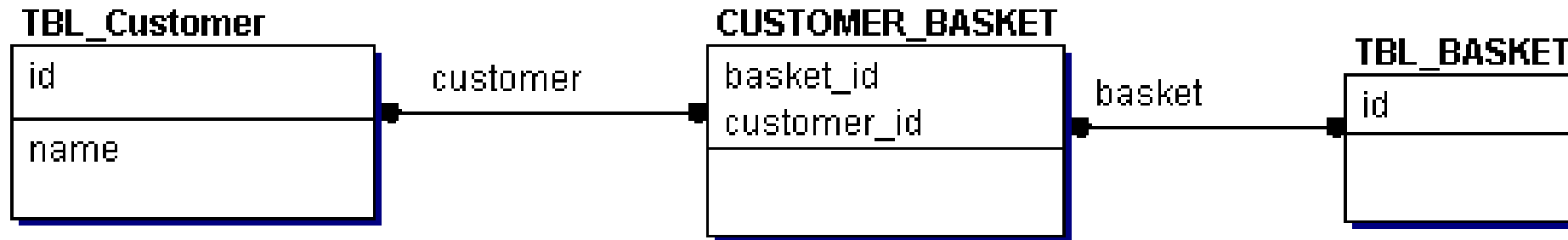
Auch für Map

```
@CollectionTable( name = "entity.attribute")  
@ElementCollection  
private Set<String> someStringColl = new  
HashSet<String>();
```





@OrderColumn



Many-To-
Many

```
@ManyToMany
@JoinTable(name="CUSTOMER_BASKET"
    ,joinColumns={@JoinColumn(name="basket_id")}
    ,inverseJoinColumns={@JoinColumn(name="customer_id")})
private List<Customer> owners;
```

Aufgabe 1_8

- Erweitern Sie die Klasse Vehicle um ein Attribut:
 - `List<Shop> locationHistory`
 - Dies solle alle Ehemaligen Standorte beinhalten
 - Die Funktion `setLocation` soll den zu ersetzenden Standort in die Historie eintragen.
 - Mappen Sie das neue Attribut auf die DB
- Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema

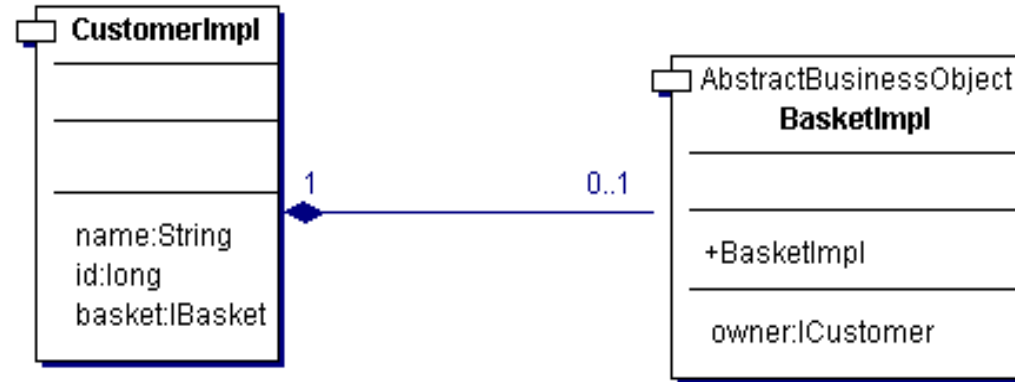




Kleines Quiz

1. Die Beziehung wird durch eine Zwischentabelle abgebildet. Könnte man die Zwischentabelle als JPA-Entität abbilden? Wie sähe diese Klasse aus?

Cascading



Cascading

- In allen Beziehungstypen
 - Attribute cascade={ Typen }
 - Enumeration CascadeType
 - `@OneToOne(cascade = CascadeType.PERSIST)`
- Reduziert EntityManager Aufrufe
- **Performance killer!!!**
 - Anzahl an DB- Anfragen

CascadeType.*

CascadeType.PERSIST

CascadeType.MERGE

CascadeType.REMOVE

CascadeType.REFRESH

CascadeType.ALL

cascade oder orphanRemoval?

Modul 1_6_1

- CascadeType.REMOVE: das Kind wird gelöscht, wenn das Elternobjekt gelöscht wird.
- orphanRemoval = true: das Kind wird gelöscht, sobald es aus der Collection entfernt wird – auch ohne remove auf dem Parent.
- CascadeType.ALL enthält REMOVE, aber niemals orphanRemoval.
- Beides gemeinsam ist der Normalfall für echte Kompositionen.

```
@OneToMany(mappedBy = "type", cascade = CascadeType.ALL, orphanRemoval = true)
private List<Vehicle> vehicles = new ArrayList<>();

type.getVehicles().remove(v);    // orphanRemoval -> DELETE FROM vehicle
em.remove(type);                 // cascade REMOVE -> DELETE aller Kinder
```

Übung: Kaskadierung und verwaiste Kinder

🕒 15 min

Level ●●●

Modul 1_6_1

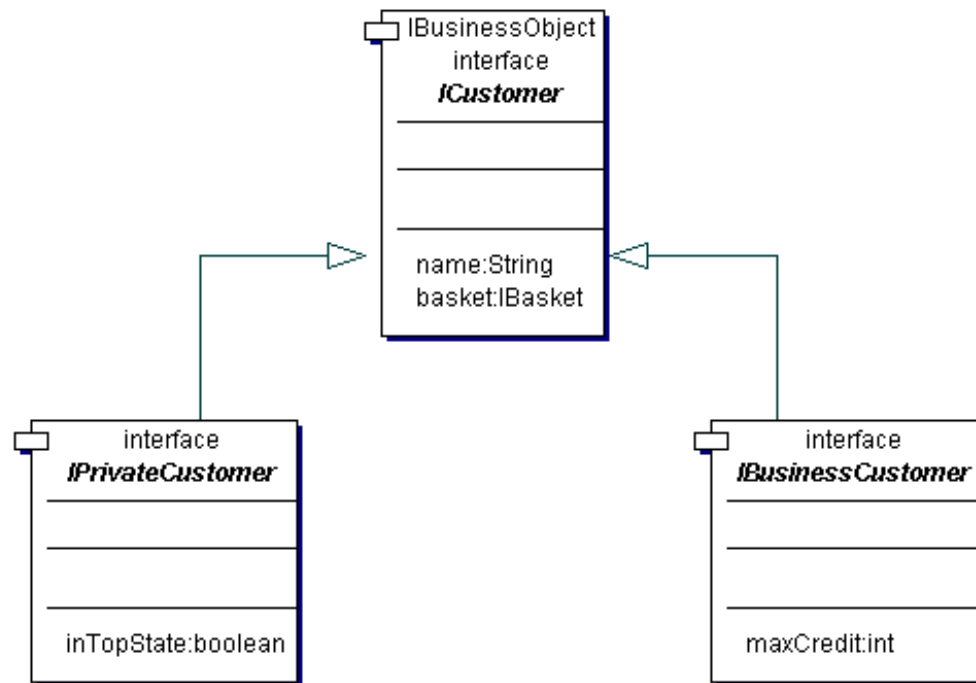
AUFTRAG

- Die Beziehung VehicleType → Vehicle mit cascade und orphanRemoval versehen.
- Erst ein Fahrzeug aus der Liste entfernen, Transaktion committen.
- Dann den kompletten VehicleType löschen.
- Vorher notieren: welche DELETE-Statements erwarten Sie, in welcher Reihenfolge?

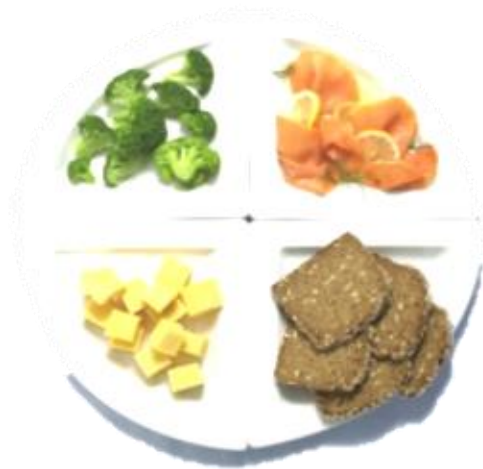
ERFOLGSKRITERIEN

- ✓ Test grün.
- ✓ Im Log erscheinen genau die vorhergesagten DELETES.
- ✓ Sie können den Unterschied in einem Satz erklären.

Stolperstein: orphanRemoval greift nur, wenn die Collection tatsächlich verändert wird – nicht beim Setzen der Gegenseite auf null.



Strategien



TBL_CUSTOMER_HIERARCHY

id
name basket_id inTopState maxCredit DISCRIMINATOR

```
@Entity
@Table (name="TBL_CUSTOMER_HIERARCHY")
@Inheritance (strategy=
    InheritanceType.SINGLE_TABLE)
@DiscriminatorColumn (name="DISCRIMINATOR"
    ,discriminatorType=DiscriminatorType.STRING)
@DiscriminatorValue ("thevalue")
```

A close-up photograph of two hands shaking in a firm grip. The hand on the left is slightly larger and has more hair on the forearm. The hand on the right is smaller. The background is plain white.

Vorteile

Nachteile

- Einfach
- Performance
 - JOINS/UNIONS

- Keine NOT NULL Spalte (Unterklassen)
- Keine Normalisierung

Aufgabe 1_9_1

- Mappen Sie die Hierarchie VehicleType, Car, Truck mittels SingleTable
 - Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema

```
@Test
public void testInheritance() {
    assertNotNull(super.manager.find(Truck.class,
    Truck.getId()));
}
```





Kleines Quiz

1. Wie sieht der Inhalt der Discriminator Spalte aus, wenn sie die Daten vom Testdurchlauf commiten?

Quiz: Warum kann SINGLE_TABLE keine NOT-NULL-Spalten für ~~Unter~~klassen haben?

- A) Weil Hibernate NOT NULL bei Vererbung generell ignoriert.
- B) Weil alle Unterklassen sich eine Tabelle teilen – die Spalte ist für die anderen Typen leer.
- C) Weil der Diskriminator die Spalte belegt.
- D) Weil JPA das in der Spezifikation verbietet.

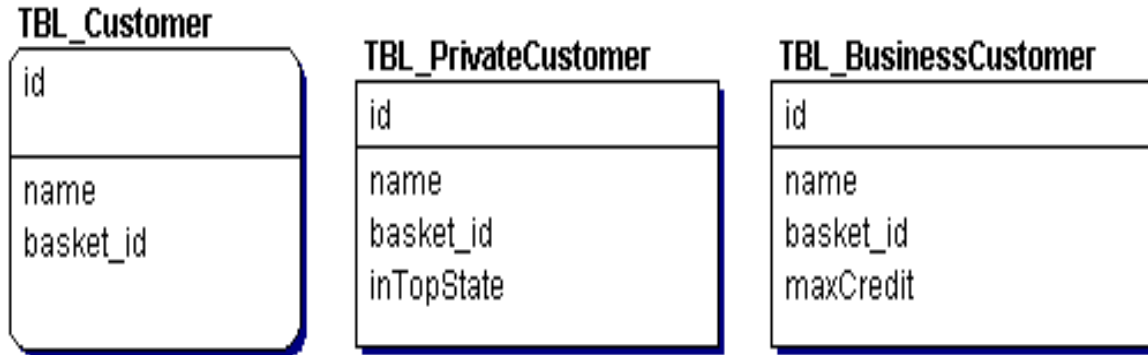
Think – Pair – Share: 1 Minute allein, 1 Minute zu zweit, dann Auflösung.

Auflösung: SINGLE_TABLE und NOT NULL

B – eine Tabelle für alle Typen, also muss jede Spezialspalte nullable sein.

- Ein Truck-Datensatz hat keine Spalte doors zu füllen – sie bleibt NULL.
- Wer NOT NULL braucht, nimmt JOINED oder prüft über Bean Validation auf Anwendungsebene.
- Gegenrechnung: SINGLE_TABLE ist die schnellste Strategie, weil kein Join nötig ist.
- Entscheidung immer entlang der häufigsten Abfrage treffen.

Table per Concrete Class



```
@Entity
@Table(name="TBL_Customer")
@Inheritance(strategy=InheritanceType.TABLE_PER_CLASS)
```

```
@Entity
@Table(name="TBL_PrivateCustomer")
```

```
@Entity
@Table(name="TBL_BusinessCustomer")
```

- Constraints

- Keine Normalisierung
- Multiple Anfragen oder Unions beim Laden

A close-up photograph of two human fists bumping in a celebratory gesture. The fists are positioned in the center of the frame, with the left fist on the left and the right fist on the right. The skin is light-toned and has some hair on the forearms. The background is plain white.

Vorteile

Nachteile

Aufgabe 1_9_2

- Mappen Sie die Hierarchie VehicleType, Car, Truck mittels TablePerClass
 - Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema

```
@Test
public void testInheritance() {

    assertNotNull(super.manager.find(Truck.class,
    Truck.getId()));
}
```

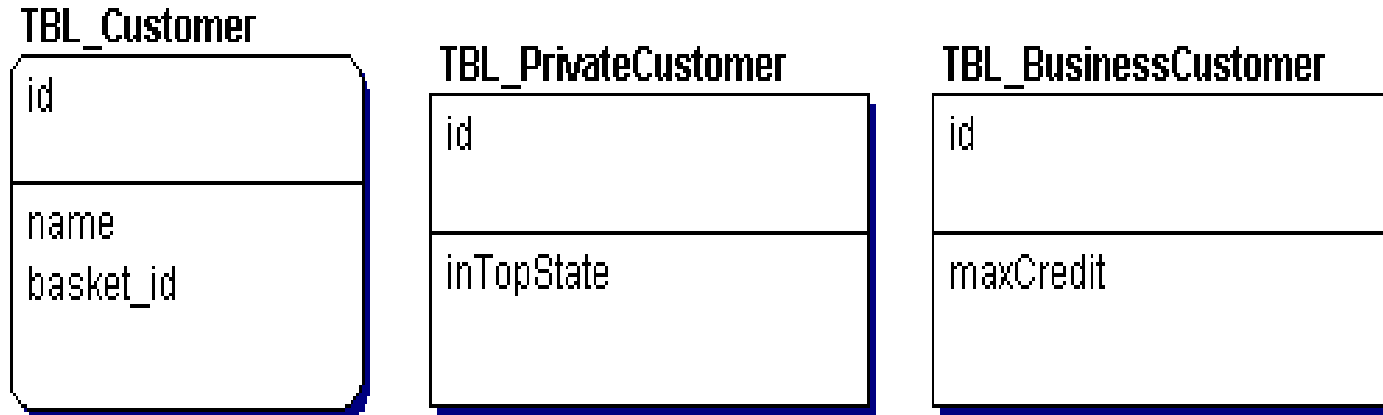




Kleines Quiz

1. Markieren Sie die Klasse `VehicleType` als abstrakt. Welche Änderungen ergeben sich im Schema?

Table per Subclass



```
@Entity
@Table(name="TBL_Customer")
@Inheritance(strategy=InheritanceType.JOINED)
```

```
@Entity
@Table(name="TBL_PrivateCustomer")
```

```
@Entity
@Table(name="TBL_BusinessCustomer")
```

Aufgabe 1_9_3

- Mappen Sie die Hierarchie VehicleType, Car, Truck mittels Joinedao
 - Testen Sie das Mapping mit der Testklasse
 - Die vorhandenen Methoden sollten ausreichen
 - Sie können aber gerne weitere erstellen
- Überprüfen Sie das erzeugte DB Schema

```
@Test
public void testInheritance() {
    assertNotNull(super.manager.find(Truck.class,
    Truck.getId()));
}
```





Kleines Quiz

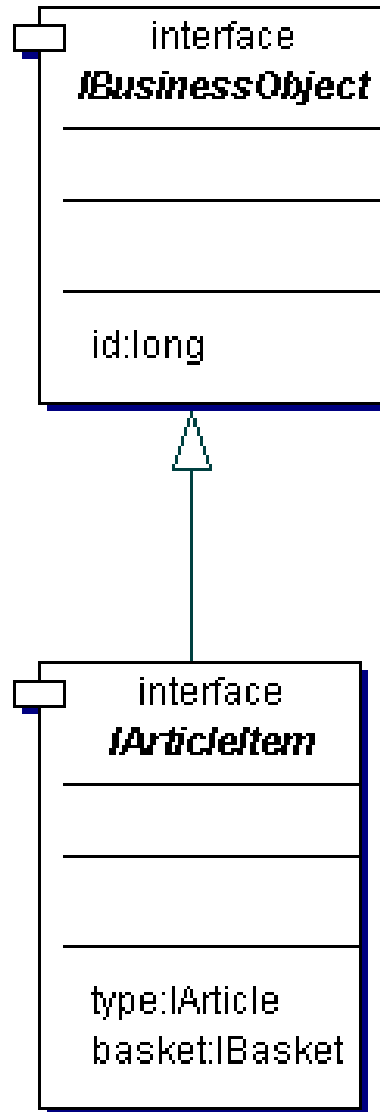
1. Markieren Sie die Klasse `VehicleType` als abstrakt. Welche Änderungen ergeben sich im Schema?

Nachteile

- Performance
- Single Table

Vorteile

- Normalisierung
- Constraints



Basisklassen

(Abstract) Superentity

Basisklasse

`@MappedSuperclass`

Unterklassen

`@Entity`

`@PrimaryKeyJoinColumn(name="NEW_PK")`

`@AttributeOverride(name="alterName"
column=@Column(name="neuerName"))`

Aufgabe 1_9_4: Teil 1

- Basisklasse erstellen: Erstelle im Paket `net.rentacar.model` eine neue abstrakte Klasse mit dem Namen `AbstractBusinessObject`.
 - Annotiere die Klasse mit `@MappedSuperclass`.
 - Verschiebe das Primärschlüssel-Feld `id` (String) aus den bestehenden Entitäten in diese Basisklasse.
 - Füge ein neues Feld `version` vom Typ `long` hinzu. Annotiere es mit `@Version`, damit JPA dieses Feld für das Optimistic Locking verwendet. Optional: Gib der Spalte über `@Column(name="optimisticLocking")` einen spezifischen Namen.
 - Logik zentralisieren: Implementiere in `AbstractBusinessObject` einen Standardkonstruktor, der automatisch eine neue UUID als ID generiert, sofern keine übergeben wurde.



Aufgabe 1_9_4: Teil 2

- Entitäten anpassen:
 - Lasse alle relevanten Entitäten (z. B. Vehicle, Person, Shop, VehicleType, Model, Nutzer) von AbstractBusinessObject erben.
 - Entferne die nun redundanten id-Felder und deren Getter aus diesen Entitäten.
 - Passe die Konstruktoren der Entitäten so unter Verwendung von super(id) an, dass sie die ID-Initialisierung an die Basisklasse delegieren.



Aufgabe 1_9_4: Erfolgskriterien

- Acceptance Criteria
 - Das Projekt lässt sich fehlerfrei kompilieren.
 - In der Datenbank erhalten alle Tabellen der betroffenen Entitäten automatisch eine zusätzliche Spalte für die Versionierung (z. B. optimisticLocking).
 - Beim Erzeugen neuer Instanzen wird automatisch eine UUID generiert, ohne dass dies in jeder Entität einzeln implementiert werden muss.



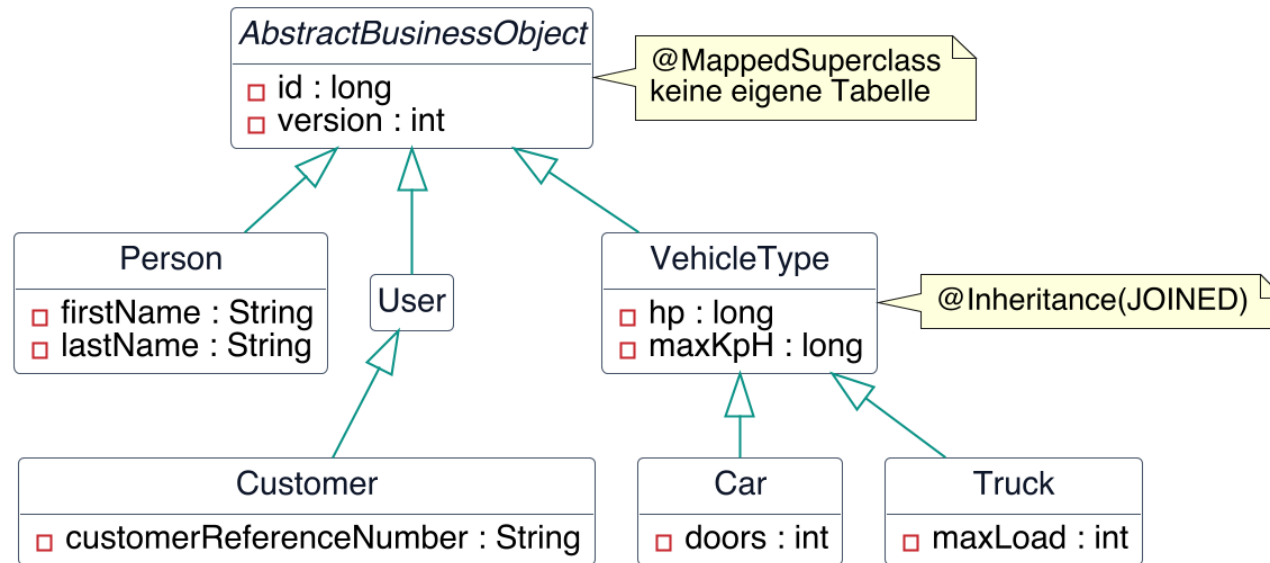


Kleines Quiz

- Abgrenzung zu Vererbungsstrategien
 - Was ist der Unterschied zu @Inheritance (SINGLE_TABLE, JOINED, TABLE_PER_CLASS)?

Vererbung im Modell

Module 1_9_1 bis 1_9_4



- **AbstractBusinessObject** ist `@MappedSuperclass`: gemeinsame Felder, aber keine eigene Tabelle und keine Abfragbarkeit.
- **VehicleType**, **Car** und **Truck** bilden dagegen eine echte Entity-Hierarchie.
- Im Projekt mit `InheritanceType.JOINED` – Module 1_9_1 bis 1_9_3 zeigen die Alternativen.
- Faustregel: `@MappedSuperclass` für technische Felder, Entity-Vererbung für fachliche Spezialisierung.

Drei Strategien – dasselbe Modell, drei Schemata

Module 1_9_1 bis 1_9_3

Strategie	Tabellen	Polymorphe Abfrage	Not-Null möglich	Einsatz
SINGLE_TABLE	eine, mit Diskriminator	sehr schnell	nein	flache Hierarchien, Lesezugriff
JOINED	eine je Klasse	Join je Ebene	ja	normalisiertes Schema, viele Felder
TABLE_PER_CLASS	eine je konkreter Klasse	UNION, langsam	ja	selten, kaum polymorphe Zugriffe

Predict-then-Run: vor dem Testlauf das erwartete CREATE TABLE aufschreiben – dann -Pinspect starten.

Recap Kapitel 2

Dieselben Lernziele – jetzt als Häkchenliste.

- ✓ Sie bilden alle vier Assoziationstypen mit korrekter Besitzseite ab.
- ✓ Sie unterscheiden cascade und orphanRemoval sicher.
- ✓ Sie erkennen die drei Vererbungsstrategien am erzeugten SQL.
- ✓ Sie wählen zwischen @MappedSuperclass und Entity-Vererbung begründet.

Als Nächstes: Abfragen schreiben – und zwar so, dass sie auch mit 100.000 Datensätzen tragen.



Q&A

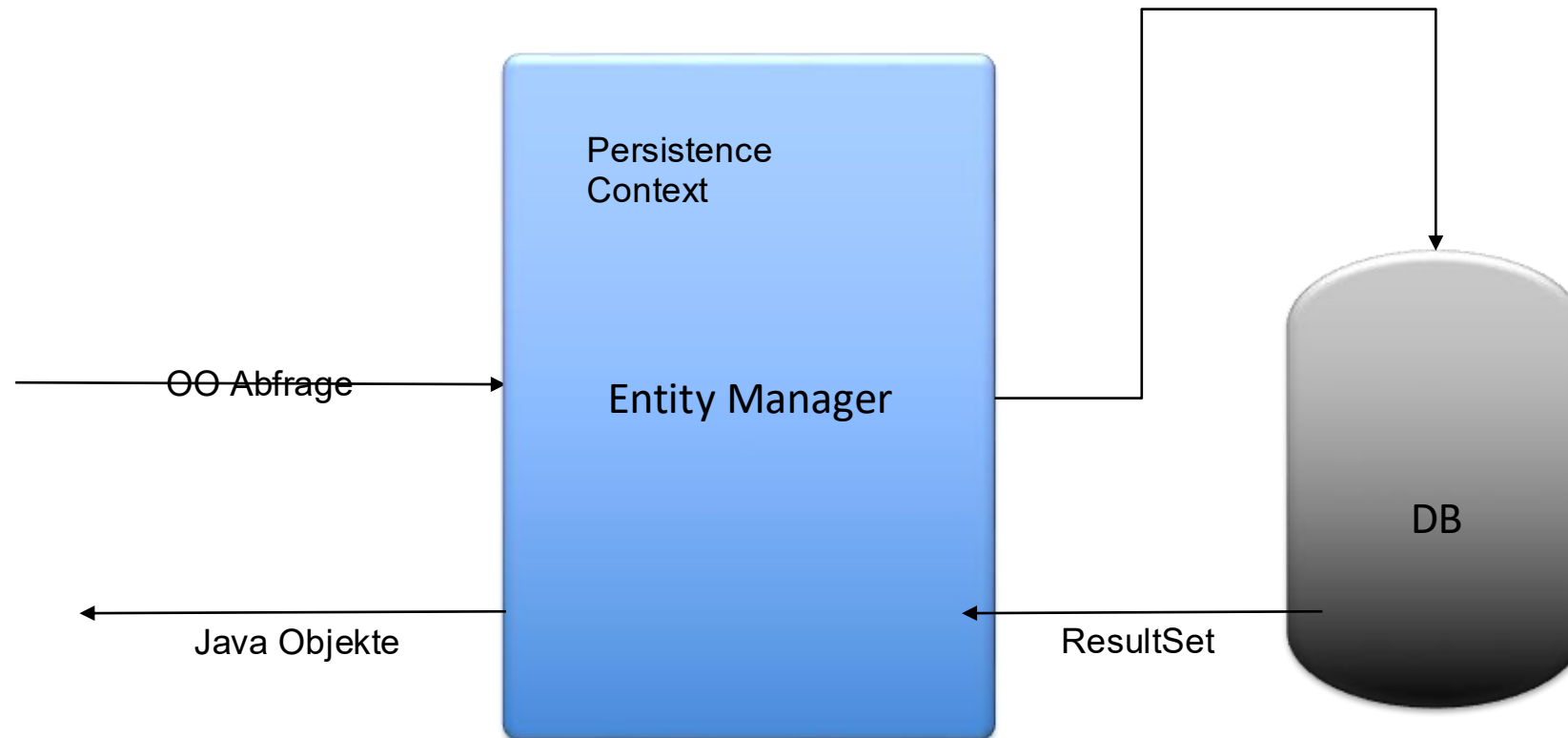
03

JPQL – Abfragen auf dem Objektmodell

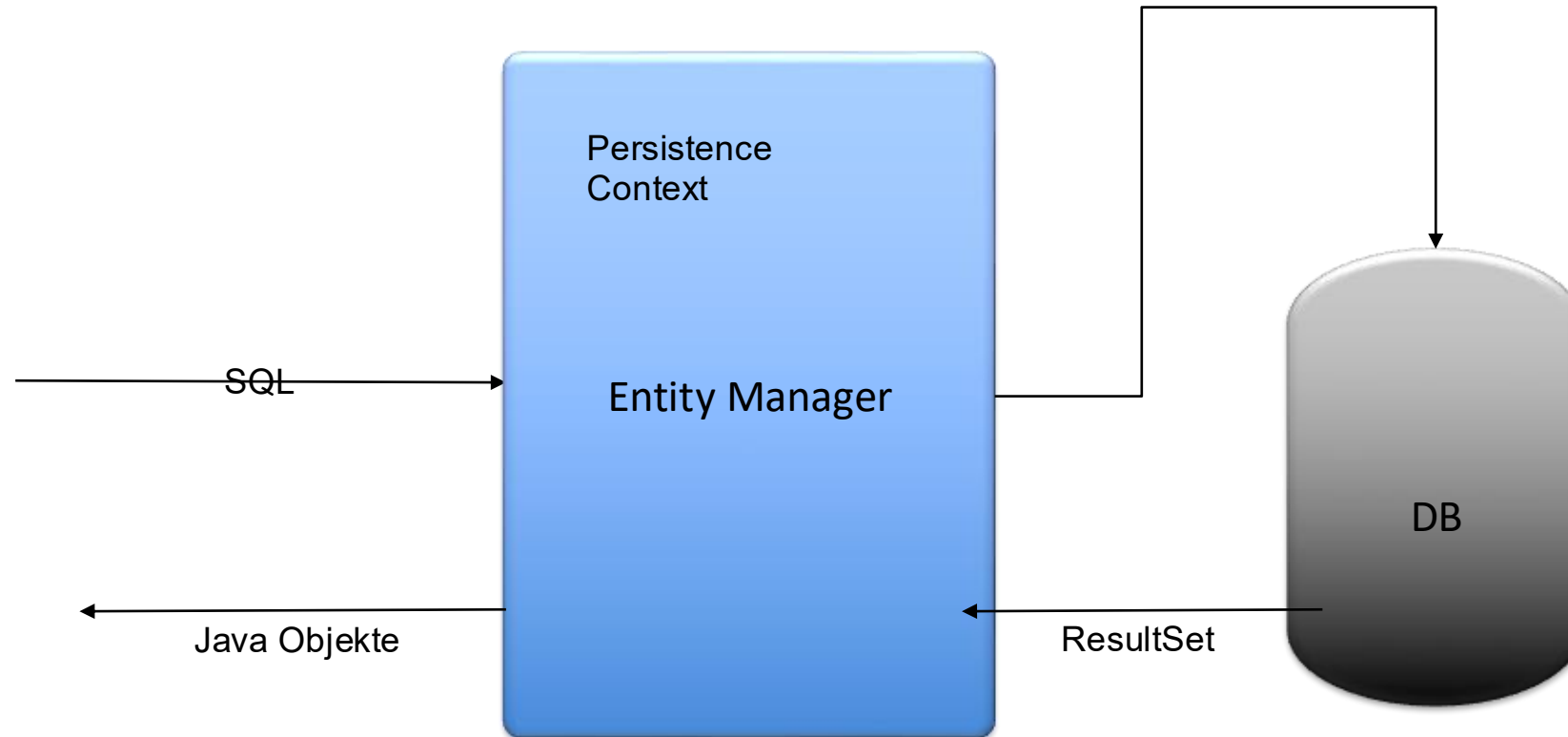
Nach diesem Kapitel können Sie ...

- JPQL-Abfragen typsicher und parametrisiert schreiben,
- Ergebnisse seitenweise und stabil sortiert liefern,
- gezielt Projektionen statt vollständiger Entities laden,
- Native und Named Queries dort einsetzen, wo sie wirklich passen.

Ablauf einer Query



Ablauf einer Native Query



Query API

EM = Factory

- TypedQuery createQuery(String queryString, Class entityClass)
- TypedQuery createNamedQuery(String name, Class entityClass)
- Query createNativeQuery(String sqlString)
- Query createNativeQuery(String sqlString, String resultSetMapping)
- Query createNativeQuery(String sqlString, Class resultClass)

Entity Manager Query API

Bezogen auf Name

- `setParameter(String name, Object value)`
- `setParameter(String name, Date value, TemporalType temporalType)`
- `setParameter(String name, Calendar value, TemporalType temporalType)`

Bezogen auf Position

- `setParameter(int position, Object value)`
- `setParameter(int position, Date value, TemporalType temporalType)`
- `setParameter(int position, Calendar value, TemporalType temporalType)`

Query API

- execute

- Resultat
 - List getResultList()
 - Object getSingleResult()
 - int executeUpdate()

Die Query-API – eine Variante genügt

Module 2_01 und 2_02

- TypedQuery<T> ist typsicher – kein Cast, kein unchecked warning.
- getResultList liefert eine Liste, getSingleResult genau ein Ergebnis.
- getSingleResult wirft NoResultException bzw. NonUniqueResultException.
- Seit JPA 3.1 gibt es getSingleResultOrNull – spart das try/catch.

```
TypedQuery<Vehicle> q =  
    em.createQuery("SELECT v FROM Vehicle v", Vehicle.class);  
  
List<Vehicle> alle = q.getResultList();  
Vehicle eines      = q.getSingleResult();      // wirft bei 0 oder >1  
Vehicle optional  = q.getSingleResultOrNull(); // JPA 3.1
```

Query Erzeugen

```
Query query = entityManager
    .createQuery("SELECT cust FROM Customer cust WHERE
        cust.id=" + id);
```

```
Query query = entityManager
    .createQuery("SELECT bask FROM Basket bask WHERE " +
        "bask.owner.id=" + id);
```

```
Query query = entityManager
    .createQuery("SELECT bask, cust FROM Basket bask,
        Customer cust");
List tuple = query.getResultList();
IBasket basket = (IBasket) ((Object[])tuple.get(0))[0];
ICustomer cust = (ICustomer) ((Object[])tuple.get(0))[1];
```

Aufgabe 2_01: Teil 1

- Wir möchten nun gezielt nach Fahrzeugtypen suchen, die eine gewisse Leistung erbringen. Deine Aufgabe ist es, eine Abfrage zu formulieren, die alle Fahrzeugtypen filtert, die mehr als 130 PS haben.
- **Aufgabe**
 - Testklasse `net.rentacar.TestQuery`
 - JPQL-Abfrage implementieren: Erweitere die Methode `testQueryForVehicleMoreThan130HP()` (oder implementiere sie), um eine Abfrage mit dem EntityManager auszuführen.
 - Nutze `manager.createQuery(String jpql)`.
 - Die Abfrage soll alle Instanzen von `VehicleType` finden, deren Attribut `hp` (Horsepower) größer als 130 ist.
 - Verwende `getResultList()`, um die Liste der Ergebnisse zu erhalten.



Aufgabe 2_01: Teil 2

```
@Test
public void
testQueryForVehicleMoreThan130HP() {
    // 1. JPQL String definieren
    String jpql = "SELECT ... FROM ... WHERE
    ...";

    // 2. Abfrage ausführen
    List<VehicleType> results =
manager.createQuery(jpql).getResultList();

    // 3. Verifizieren
    assertEquals(1, results.size());
}
```





Kleines Quiz

1. Was passiert, wenn du in deiner JPQL-Abfrage den Tabellennamen aus der Datenbank (z. B. `tbl_VehicleType`) anstatt des Klassennamens (`VehicleType`) verwendest?
2. Warum ist in JPQL ein Alias (z. B. `SELECT v FROM VehicleType v`) im Gegensatz zu Standard-SQL fast immer verpflichtend?
3. Welchen Datentyp haben die Elemente in der Liste, die `getResultList()` zurückgibt, wenn du `SELECT v FROM VehicleType v` ausführst? Werden hier echte Car- oder Truck-Objekte zurückgegeben, wenn diese in der Datenbank vorhanden sind?

Parameter binden – nicht konkatenieren

Modul 2_02

Anti-Pattern

```
String jpql =  
    "SELECT u FROM User u WHERE u.id = " + id;  
  
// - kein Query-Plan-Cache  
// - Datum- und Dezimalformate brechen  
// - lehrt den Stil, der in SQL zur  
//   Injection-Lücke wird
```

So machen wir es

```
em.createQuery(  
    "SELECT u FROM User u WHERE u.id = :id",  
    User.class)  
    .setParameter("id", id)  
    .getSingleResult();  
  
// benannte Parameter sind lesbar  
// und in Named Queries prüfbar
```

Benannte Parameter sind der Standard – Positionsparameter nur, wenn eine Bibliothek sie verlangt.

WHERE

Operatoren & Wildcards

- =, <>, >,<,>=,<=
- like
- (NOT) BETWEEN
- (NOT) IN, (NOT) MEMBER OF
- IS (NOT) EMPTY, IS (NOT) NULL

```
Query query = entityManager.createQuery("SELECT cust FROM  
Customer" + "cust WHERE cust.id = ?1");  
query.setParameter(1,id);
```

```
Query query = entityManager.createQuery("SELECT bask FROM  
Basket " + "basket WHERE basket.owner = :vergleich");  
query.setParameter("vergleich",customer);
```

Aufgabe 2_02

- Wir möchten nun gezielt nach Fahrzeugtypen suchen, die eine gewisse Leistung erbringen. Deine Aufgabe ist es, eine Abfrage zu formulieren, die alle Fahrzeugtypen filtert, die mehr als 130 PS haben.
- **Aufgabe**
 - Wie 2_01 nur mittels Nutzung eines NamedParameter für hp





Kleines Quiz

1. Wie müsste die Abfrage lauten, wenn du nicht nur nach der Leistung ($hp > 130$), sondern zusätzlich nur nach Fahrzeugtypen suchen möchtest, deren $maxKpH$ über 200 liegt?
2. Was passiert wohl, wenn man das `SELECT v` durch `SELECT v.hp` ersetzt? Was wäre dann der Rückgabetyt der Liste?

Paging: setFirstResult und setMaxResults

Modul 2_03

- setFirstResult ist der Offset, setMaxResults die Seitengröße.
- Die Methode heißt setFirstResult – nicht setFirstIndex.
- Ohne ORDER BY ist die Reihenfolge undefiniert: Seite 2 kann Einträge von Seite 1 wiederholen.
- Bei JOIN FETCH auf eine Collection paginiert der Provider im Speicher – Warnung im Log beachten.

```
TypedQuery<Vehicle> q = em.createQuery(
    "SELECT v FROM Vehicle v ORDER BY v.id", Vehicle.class);
q.setFirstResult(20);    // Offset
q.setMaxResults(10);    // Seitengröße
List<Vehicle> seite3 = q.getResultList();
```

Übung: Seitenweise Fahrzeugliste

🕒 10 min

Level ●○○

Modul 2_03

AUFTRAG

- Die Fahrzeuge seitenweise ausgeben: 10 pro Seite, zweite Seite anzeigen.
- Stabile Sortierung sicherstellen.
- Im SQL-Log prüfen, wie Derby das Paging umsetzt.
- Danach ORDER BY entfernen und den Test mehrfach laufen lassen.

ERFOLGSKRITERIEN

- ✓ Test grün.
- ✓ Im Log erscheint OFFSET / FETCH FIRST.
- ✓ Sie können erklären, warum Paging ohne ORDER BY unzuverlässig ist.

JPA QL

Sonstiges

```
Query query = entityManager
    .createQuery("SELECT cust FROM Customer cust");
query =
    query.setHints("org.hibernate.timeout", 200
        );
List result = query
    .setFlushMode(FlushModeType.COMMIT)
    .getResultList();
```

Einfache Anfragen

```
Query query = entityManager.createQuery("SELECT cust.name"  
    + " FROM Customer cust");
```

```
Query query = entityManager.createQuery("SELECT c.name,"  
    + " c.id FROM Customer c");
```

```
Query query = entityManager.  
    createQuery("SELECT new de.test.IdNameWrapper(c.name, c.id) " +  
        "FROM Customer c");
```

```
Query query = entityManager.  
    createQuery("SELECT c.basket FROM Customer c");
```

Aufgabe 2_04: Teil 1

- Unser Fuhrpark-System wird komplexer. Wir müssen nun Abfragen erstellen, die Nutzergruppen filtern und den Status unserer Shops (Fuhrpark-Belegung) prüfen.
- **Aufgabe** Testklasse TestQuery
 - Filtern mit IN: Implementiere die Methode `testQueryForNutzerWithPersonnameISMichaelOrMathias()`.
 - Suche nach allen User-Objekten, deren zugehörige Person einen Vornamen hat, der in der Liste ['Michael', 'Mathias'] enthalten ist.
 - Nutze einen benannten Parameter (z. B. `:names`), um die Liste zu übergeben.



Aufgabe 2_04: Teil 2

- Prüfen auf leere Listen mit IS EMPTY: Implementiere die Methode `testQueryForShopMitKeinemVehicle()`.
 - Finde alle Shop-Objekte, die derzeit keine Fahrzeuge (carpool) besitzen.
 - Nutze dafür den IS EMPTY-Operator auf dem entsprechenden Collection-Attribut der Shop-Klasse.
- Negierung mit NOT IN: Implementiere die Methode `testQueryForShopsichtInStuttgartUndMuenchen_MitINOperator()`.
 - Finde alle Shop-Objekte, deren Standort (location) nicht in der Liste ['Muenchen', 'Stuttgart'] liegt.
 - Verwende hierfür die Kombination NOT IN.



Aufgabe 2_04: Erfolgskriterien

- Alle drei Testmethoden in TestQuery laufen grün.
- Du verwendest Parameter-Binding (setParameter) für die Listen-Werte bei den IN-Abfragen.
- Bei der IS EMPTY-Abfrage wird kein manueller Join verwendet





Kleines Quiz

1. Warum verwendet man bei der Prüfung auf eine leere Fahrzeugliste (carpool) in der Entität Shop den Operator IS EMPTY und nicht IS NULL (also WHERE s.carpool IS NULL)?
2. Stell dir vor, du hast eine Liste mit 5.000 Vornamen, die du dem IN-Operator als Parameter übergibst. Warum könnte dies problematisch sein, und was wäre eine bessere Alternative in der Datenbank-Programmierung?

DTO-Projektionen: nur laden, was gebraucht wird

Modul 2_05

- SELECT new mit voll qualifiziertem Klassennamen und passendem Konstruktor.
- Es werden nur die genannten Spalten selektiert – spürbar bei breiten Tabellen.
- Das Ergebnis ist bewusst nicht managed: kein Dirty Checking, kein Lazy Loading.
- Ideal für Listen, Exporte und REST-Antworten.

```
SELECT new net.rentacar.dto.VehicleInfo(v.id, v.licensePlate, t.model.brand)
FROM   Vehicle v
JOIN   v.type t
WHERE  t.hp > :minHp
```

Übung: Ergebnisliste als DTO

🕒 15 min

Level ●●○

Modul 2_05

AUFTRAG

- Eine DTO-Klasse mit den benötigten Feldern und passendem Konstruktor anlegen.
- JPQL mit SELECT new schreiben.
- Im Log vergleichen, welche Spalten jetzt selektiert werden.
- Versuchen, ein DTO zu ändern und zu committen – was passiert?

ERFOLGSKRITERIEN

- ✓ Test grün.
- ✓ Das SELECT enthält nur noch die drei benötigten Spalten.
- ✓ Sie können erklären, warum Änderungen am DTO nicht persistiert werden.

INNER JOINS

```
Query query = entityManager
    .createQuery("SELECT item FROM Basket AS b, "
        + " IN (b.articleItems) item WHERE item.name = ?1");
```

```
Query query = entityManager
    .createQuery("SELECT items FROM Basket AS b "
        + " INNER JOIN b.articleItems items");
```

```
Query query = entityManager
    .createQuery("SELECT items FROM Customer AS c, "
        + " IN (c.baskets) b, IN (b.articleItems) items");
```

LEFT (OUTER) JOINS

```
Query query = entityManager  
    .createQuery("SELECT c.name, b.id FROM Customer c  
    LEFT JOIN c.baskets b");
```

```
Query query = entityManager  
    .createQuery("SELECT c.name, b.id FROM Customer  
    c LEFT OUTER JOIN c.baskets b");
```

Aufgabe 2_06: Teil 1

- Wir möchten herausfinden, welche unserer Shops derzeit Fahrzeuge einer bestimmten Marke im Fuhrpark haben. Da die Beziehung von Shop zu Vehicle (carpool) eine Liste ist, müssen wir einen Join verwenden, um in dieser Liste suchen zu können.
- **Aufgabe** Testklasse TestQuery
 - testInnerJoin_SearchShopWithVehicleOfBrandVW().
 - Schreibe eine JPQL-Abfrage, die alle Shop-Objekte findet, die mindestens ein Fahrzeug in ihrem carpool haben, dessen Marke (brand) den Wert 'VW' hat.
 - Verwende die Syntax INNER JOIN f.carpool fz.
 - Nutze getSingleResult(), um das Ergebnis abzurufen, und verifiziere mit assertEquals, dass der Standort des gefundenen Shops 'Stuttgart' ist.



Aufgabe 2_06: Teil 2

- Suche mit der IN-Syntax: Implementiere die Methode `testIn_SearchShopWithVehicleOfBrandVW()`.
 - Löse die gleiche Aufgabe wie oben, aber verwende diesmal die alternative JPQL-Syntax für Joins über Collections: `FROM Shop f, IN(f.carpool) fz`.
 - Verifiziere auch hier, dass der korrekte Shop gefunden wird.



Aufgabe 2_06: Erfolgskriterien

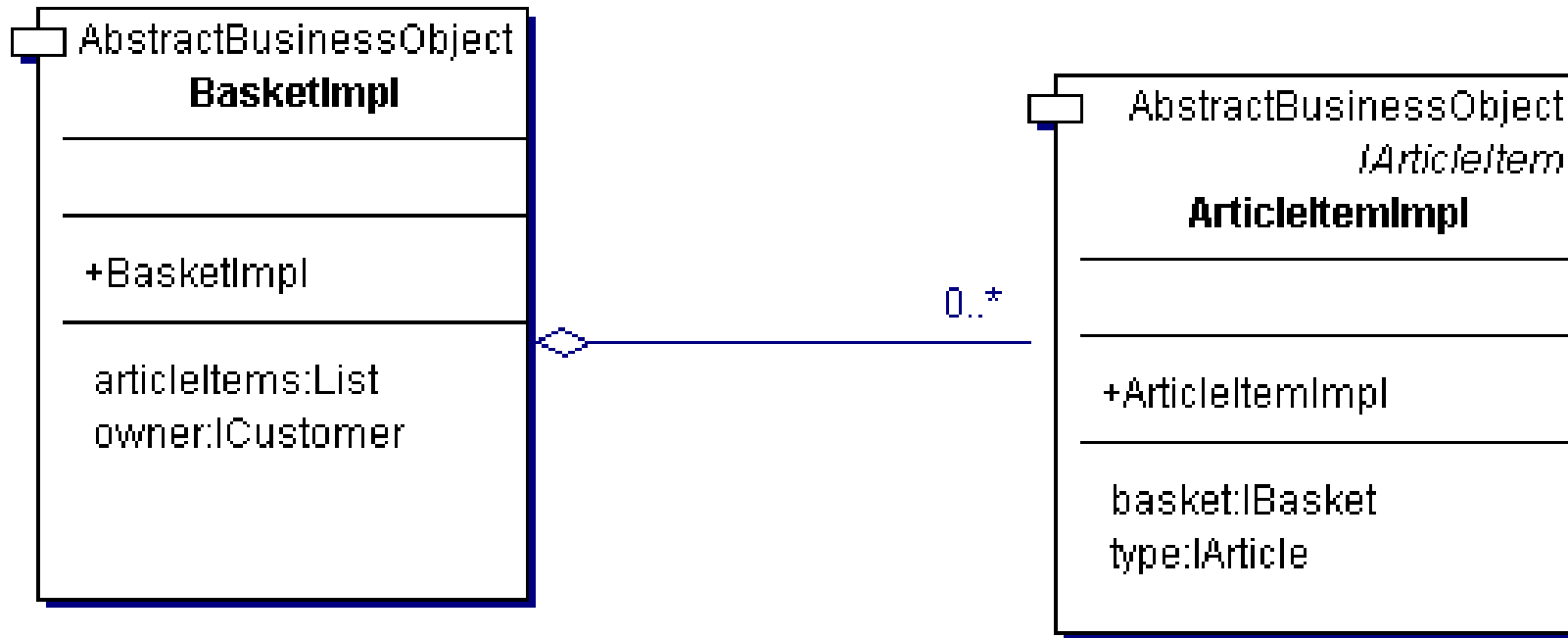
- Beide Testmethoden laufen grün.
- Du hast verstanden, wie man über eine Collection navigiert, um Attribute der darin enthaltenen Objekte zu filtern.
- Du kannst den Unterschied zwischen der Navigation über ein Einzelobjekt (f.location) und über eine Collection (JOIN f.carpool) erklären.
- Wichtiger Hinweis: In dieser Übung erwarten wir genau ein Ergebnis. Wenn eine Abfrage mehrere Shops finden würde, würde `getSingleResult()` eine Exception werfen. In der Realität würde man hier eher `getResultList()` verwenden. Für den Test nutzen wir hier die Annahme, dass unser Test-Setup nur einen passenden Shop bereithält.





Kleines Quiz

1. Warum funktioniert die Abfrage `SELECT s FROM Shop s WHERE s.carpool.type.model.brand = 'VW'` nicht, während `SELECT v FROM Vehicle v WHERE v.type.model.brand = 'VW'` problemlos funktioniert?
2. Stell dir vor, du hast einen Shop in "Berlin" angelegt, der aber noch gar keine Fahrzeuge in seinem carpool hat. Erscheint dieser Shop im Ergebnis der Abfrage `SELECT s FROM Shop s INNER JOIN s.carpool fz`, wenn man die WHERE-Klausel weglässt?



Performance
Probleme?

```

List<IBasket> baskets = entityManager
    .createQuery("FROM Basket b").getResultList();

for (IBasket basket : baskets)
{
    for (IArticleItem item :
        basket.getArticleItems()) {
        item.getId();
    }
}
  
```

Funktionen

<i>Funktion</i>	<i>Beschreibung</i>
AVG	Mittelwertsberechnung
COUNT	Berechnet die Anzahl Element in der Gruppe
MIN, MAX	Bestimmt das (Min-)Maximum der Elemente in der Gruppe
SUM	Bildet die Summe der Elemente in der Gruppe
ABS	Absolute Wert einer Zahl
SQRT	Wurzel
MOD	Modulo
LOWER,UPPER	Kleinbuchstaben, Großbuchstaben
TRIM	Abschneiden eines bestimmten Zeichens am Anfang und Ende
CONCAT	Konkationation zweier Zeichenketten
LENGTH	Länge einer Zeichenkette
LOCATE	Position eines Zeichens in einer Zeichenkette
SUBSTRING	

Gruppierungen

```
Query query = entityManager
    .createQuery("SELECT c.name, COUNT(items) "
        + "FROM Customer AS c, IN(c.basket.articleItems) " + "items"
        + "GROUP BY c.name");

Query query = entityManager
    .createQuery("SELECT c.name, COUNT(items) "
        + "FROM Customer AS c, IN(c.basket.articleItems) " + "items"
        + "GROUP BY c.name HAVING COUNT(items) > 5");
```

Aufgabe 2_08: Teil 1

- Die Geschäftsführung möchte eine Statistik darüber, wie oft die verschiedenen Fahrzeugmodelle reserviert wurden. Dabei interessieren uns besonders die Modelle, die eine hohe Auslastung haben.
- **Aufgabe** Testklasse TestQuery#testGroupBy().
 - Erstelle eine JPQL-Abfrage, die den Modellnamen des Fahrzeugs und die Anzahl der Reservierungen für dieses Modell zurückgibt.
 - Du musst dafür über Customer und deren reservations navigieren.
 - Das Ergebnis der Abfrage ist eine List<Object[]>. Jedes Object[] enthält an Index 0 den Modellnamen (String) und an Index 1 die Anzahl (Long).



Aufgabe 2_08: Teil 2

- Gruppierung mit Filterung (HAVING): Implementiere die Methode `testGroupByAndHaving()`.
 - Erweitere die vorherige Abfrage. Wir möchten nur die Modelle sehen, die mehr als 2 Reservierungen haben.
 - Nutze dafür die HAVING-Klausel.



Aufgabe 2_08: Erfolgskriterien

- Beide Testmethoden laufen grün.
- Du hast verstanden, dass im SELECT bei einer Gruppierung nur Felder stehen dürfen, nach denen auch gruppiert wird (oder die in einer Aggregatfunktion stehen).
- Du kannst den Unterschied zwischen WHERE und HAVING erklären.





Kleines Quiz

1. Wenn du `SELECT count(res) FROM Reservation res` ausführst, was ist der Rückgabetypp von `getSingleResult()`? Ist es ein Integer, ein Long oder ein int?
2. Könnte man die Bedingung `HAVING count(res) > 2` auch einfach in die `WHERE`-Klausel schreiben? Warum (nicht)?

Unterabfragen

```
Query query = entityManager
    .createQuery("SELECT c.name FROM Customer AS c"
        + "WHERE 5 < (SELECT COUNT(it) FROM"
        + "c.basket.items it)");
```

Aufgabe 2_09: Teil 1

- In unserem Autovermietungssystem (Rent-a-Car) möchten wir gezielte Analysen über das Buchungsverhalten unserer Kunden durchführen. Ein wichtiger Aspekt ist die Identifizierung von "Premium-Kunden", die bereits einen signifikanten Umsatz generiert haben. Da die Umsatzdaten (Preise) in den einzelnen Reservierungen gespeichert sind, müssen wir diese für jeden Kunden aggregieren und filtern.
- In dieser Übung lernst du, wie du Subqueries (Unterabfragen) in JPQL einsetzt, um komplexe Filterbedingungen zu formulieren, die auf aggregierten Daten von verknüpften Entitäten basieren



Aufgabe 2_09: Teil 2

- Ergänzen Sie die Testmethode testSubQuery in der Klasse TestQuery, um folgende Abfrage mittels JPQL zu implementieren:
- **Ziel:** Finden Sie alle Kunden (Customer), deren Gesamtsumme aller Reservierungspreise einen bestimmten Schwellenwert überschreitet.
- **Anforderung:** Verwenden Sie eine JPQL-Abfrage mit einer Subquery.
- Die Subquery soll die Summe (SUM) der Preise (price) aller Reservierungen (reservations) berechnen, die einem Kunden zugeordnet sind.
- Filtern Sie die Kunden so, dass nur diejenigen zurückgegeben werden, bei denen diese Summe größer als 200 ist.
- Das Ergebnis soll eine Liste von Customer-Objekten sein.



Aufgabe 2_09: Erfolgskriterien

- Korrekte JPQL-Syntax: Die Abfrage verwendet eine korrekte Unterabfrage im WHERE-Teil.
- Aggregatfunktion: In der Subquery wird die SUM()-Funktion korrekt auf das Preisfeld der Reservierungen angewendet.
- Filterung: Es werden nur Kunden zurückgegeben, deren kumulierter Reservierungswert über 200 liegt.
- Test-Pass: Die Assertion assertEquals(2, resultList.size()) wird erfolgreich ausgeführt (basierend auf den in setUp vorbereiteten Testdaten).
- Typisierung: Die Ergebnismethode liefert die erwartete Liste von Entitäten zurück.

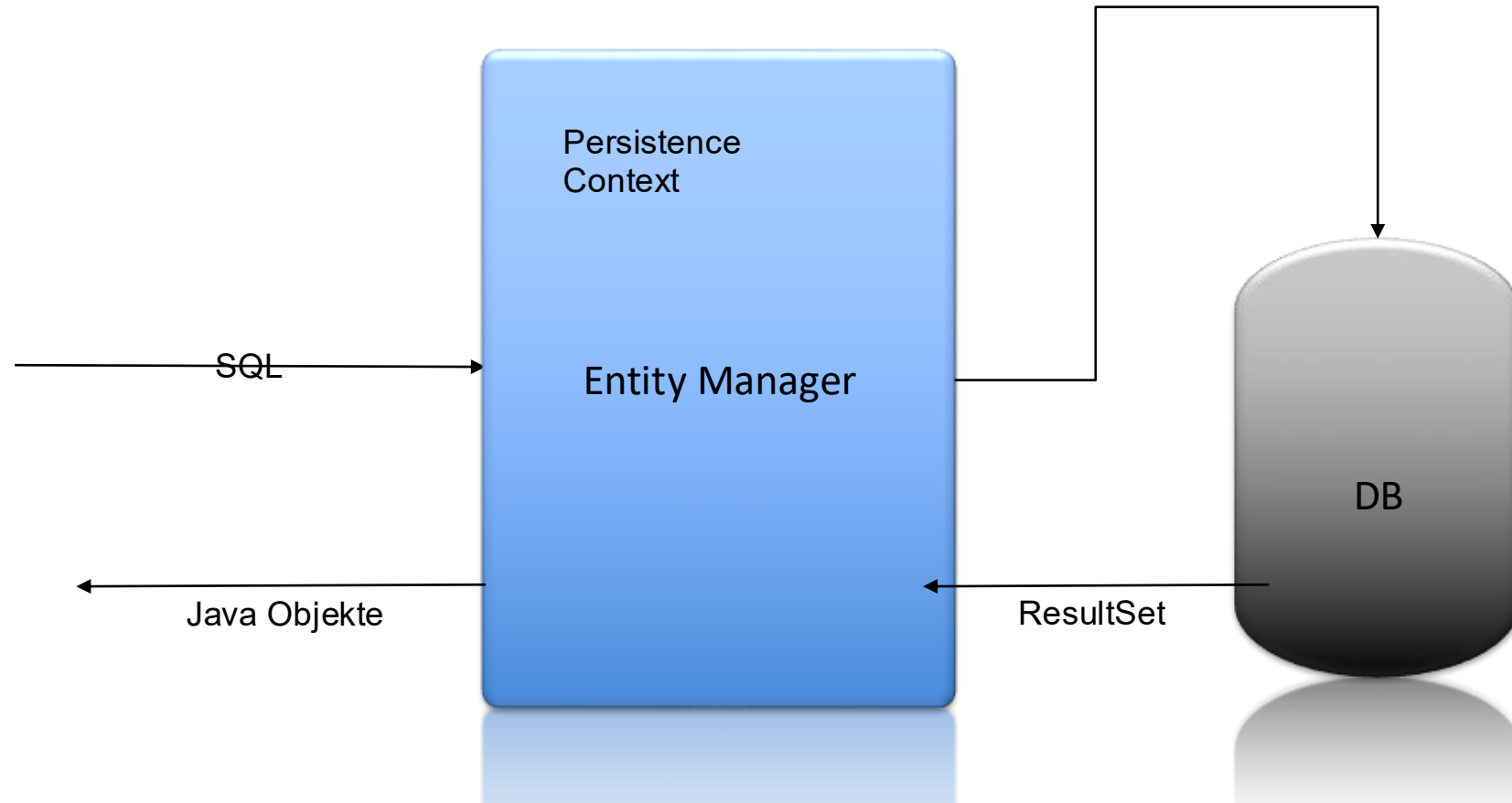




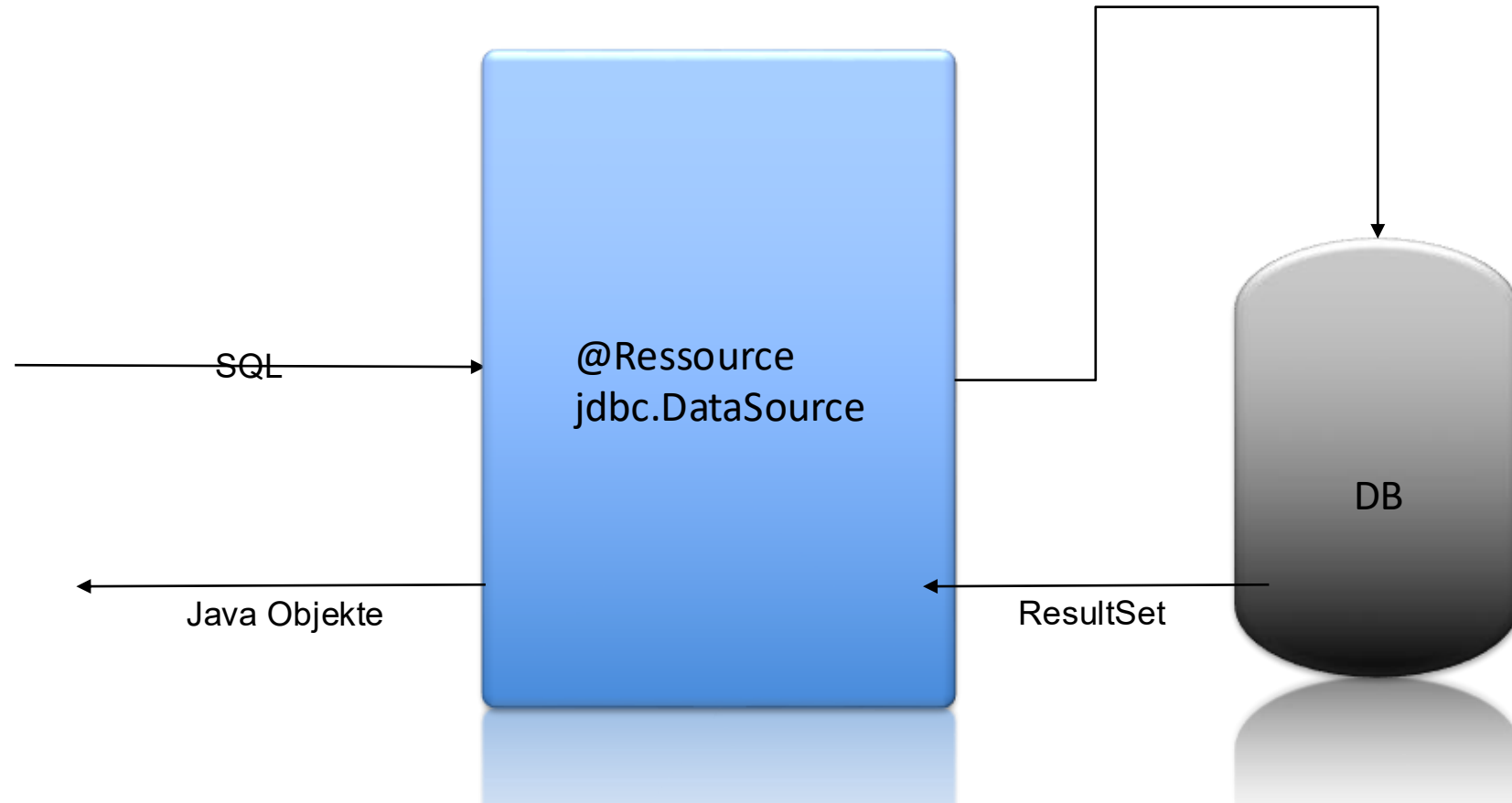
Kleines Quiz

1. Was ist der entscheidende Unterschied in der Formulierung einer Subquery, wenn man sie auf eine Collection-Variable des äußeren Objekts (wie im Beispiel k.reservations) bezieht, im Vergleich zu einer völlig unabhängigen Entität?
 - a) Hintergrund: Denken Sie an die Identifizierung des Objekts in der Subquery. Wie "weiß" die Subquery, welche Reservierungen sie summieren soll?
2. In welchen Teilen einer JPQL-Abfrage dürfen Subqueries laut JPA-Standard verwendet werden?
 - a) Nur in der WHERE-Klausel.
 - b) Nur in der WHERE- und HAVING-Klausel.
 - c) In der SELECT-, WHERE- und HAVING-Klausel.
 - d) In allen Klauseln, inklusive der FROM-Klausel

Native Query



Native Query 2

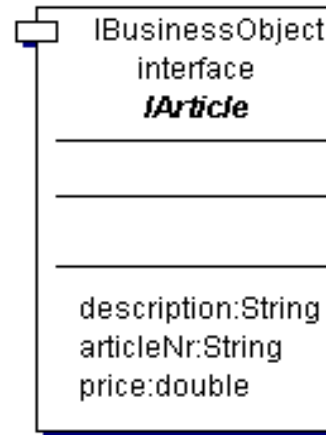


Entity Manager

EntityManager API - Query

- Query createQuery(String queryString)
- Query createNamedQuery(String name)
- **Query createNativeQuery(String sqlString)**
- **Query createNativeQuery(String sqlString, String resultSetMapping)**
- **Query createNativeQuery(String sqlString, Class resultClass)**

Native Queries mit ResultClass



```
Query query = entityManager
    .createNativeQuery("SELECT a.id, a.description,
        " "a.articleNr, a.price FROM TBL_Article a",
        Article.class);
```

Native Queries mit ResultSetMapping

```
@Entity
@SqlResultSetMapping(name =
    "articleItemAndArticleMapping",
    entities={@EntityResult(entityClass= Article.class),
        @EntityResult(entityClass= ArticleItem.class,
            fields={@FieldResult(name= "id",column= "IT_ID")
                })})
    )
```

```
Query query = entityManager
    .createNativeQuery("SELECT it.id AS IT_ID,
        it.basket_id, " + "it.article_id, a.id, a.articelNr
        FROM TBL_Article a, " + "TBL_ArticleItem it WHERE
        it.article_id = a.id",
        "articleItemAndArticleMapping");
```

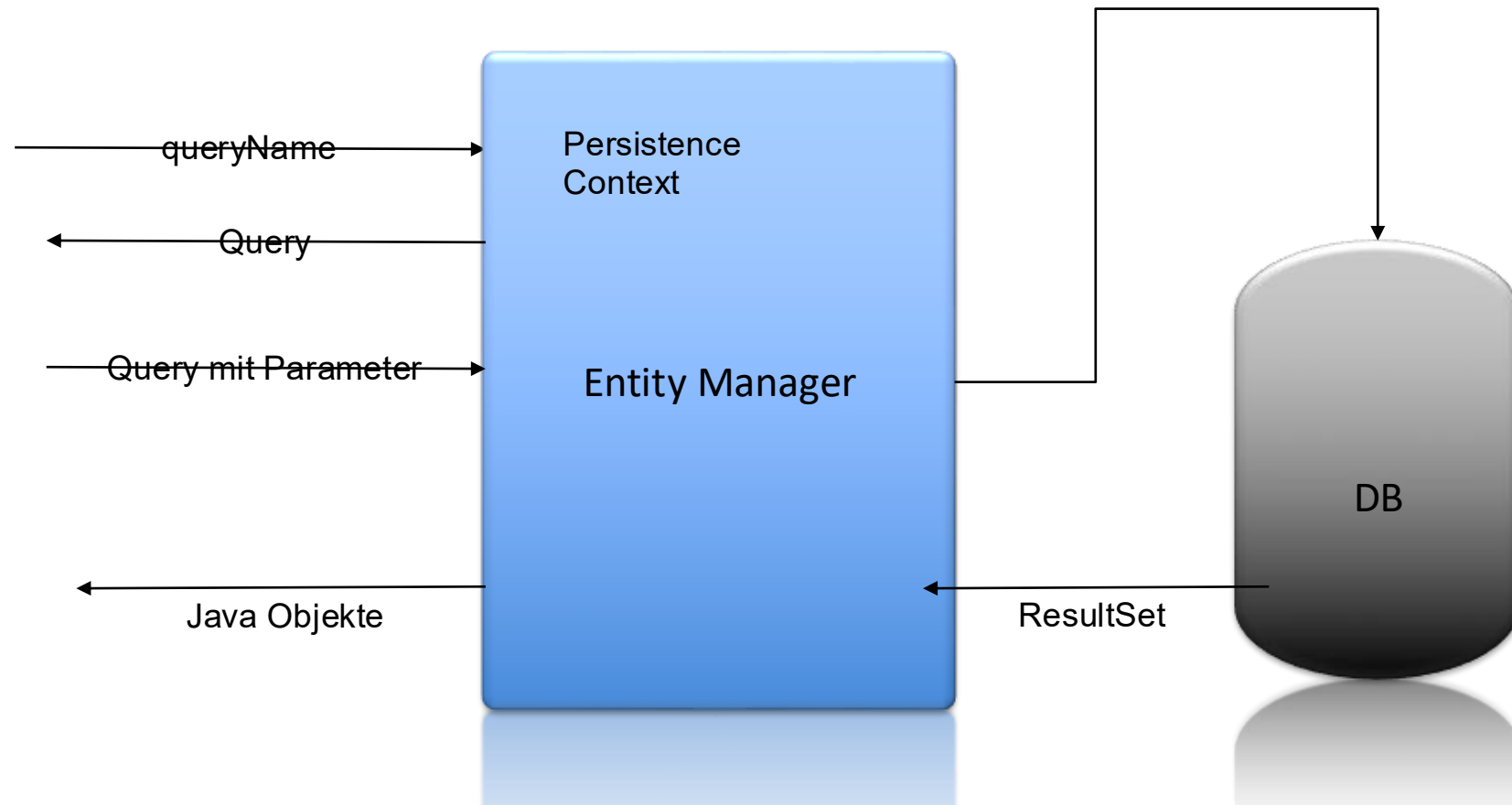
Native Queries – wenn JPQL nicht reicht

Modul 2_10

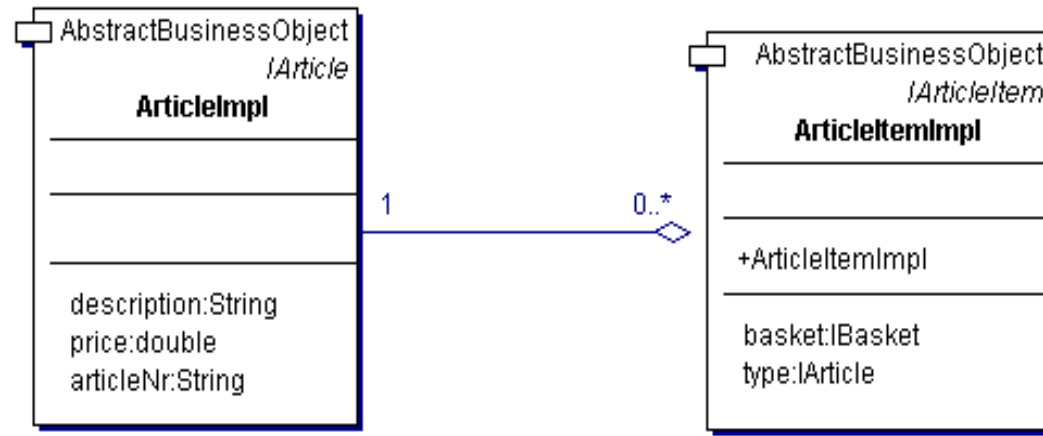
- Echtes SQL: Datenbankfunktionen, Hints, Recursive CTEs.
- Mit Result-Class oder @SqlResultSetMapping auf Entities abbilden.
- Preis: Bindung an den SQL-Dialekt, kein automatisches Dialect-Handling.
- Wichtig: Native Queries gehen am First-Level-Cache vorbei – vorher flushen, danach ggf. clear.

```
List<Vehicle> l = em.createNativeQuery(  
    "SELECT * FROM vehicle WHERE license_plate LIKE ?1", Vehicle.class)  
    .setParameter(1, "M-%")  
    .getResultList();
```

Named Query



Named Query



```
@NamedQuery(name =  
    "getArticleItemsByArticle",  
    query = "FROM ArticleItem it WHERE it.type = " +  
            ":article")
```

Named Queries – früh validiert, wiederverwendbar

Modul 2_11

- An der Entity deklariert, dadurch zentral auffindbar.
- Beim Start der Persistence Unit geparkt: Tippfehler fallen sofort auf, nicht erst zur Laufzeit.
- Immer mit SELECT-Klausel schreiben – die Kurzform FROM ... ist HQL, nicht JPQL.
- Gleiche Parameterbindung wie bei dynamischen Queries.

```
@Entity
@NamedQuery(name = "Vehicle.byType",
            query = "SELECT v FROM Vehicle v WHERE v.type = :type")
public class Vehicle { ... }

em.createNamedQuery("Vehicle.byType", Vehicle.class)
    .setParameter("type", golf).getResultList();
```

Recap Kapitel 3

Dieselben Lernziele – jetzt als Häkchenliste.

- ✓ Sie schreiben JPQL typsicher mit TypedQuery und benannten Parametern.
- ✓ Sie liefern Ergebnisse seitenweise und stabil sortiert.
- ✓ Sie laden mit DTO-Projektionen nur die benötigten Spalten.
- ✓ Sie wählen bewusst zwischen dynamischer, Named und Native Query.

Als Nächstes: Die häufigste Performance-Falle in Produktion: das N+1-Problem.

04

Fetch-Strategien, N+1 und Caching

Nach diesem Kapitel können Sie ...

- die Default-Fetch-Typen benennen und ihre Folgen einschätzen,
- ein N+1-Problem im SQL-Log zweifelsfrei erkennen,
- zwischen Fetch Join, Entity Graph und Eager begründet wählen,
- eine LazyInitializationException erklären und beheben.

FetchType – die Defaults kennen

Modul 2_07

Beziehung	Default	Bedeutung	Praxis
@ManyToOne	EAGER	wird immer mitgeladen	meist auf LAZY umstellen
@OneToOne	EAGER	wird immer mitgeladen	LAZY nur mit optional = false zuverlässig
@OneToMany	LAZY	erst beim Zugriff	so lassen, gezielt per Query laden
@ManyToMany	LAZY	erst beim Zugriff	so lassen

LAZY ist eine Bitte, keine Garantie: der Zugriff muss innerhalb eines offenen Persistence Context erfolgen.

Lazy Loading und Fetch Joins

```
@OneToMany(fetch = FetchType.LAZY)
private List<IArticleItem> articleItems;
```

```
Query query = entityManager
    .createQuery("SELECT b FROM Basket b,"
        + " LEFT JOIN FETCH b.articleItems");
```

Das N+1-Problem – sichtbar gemacht

Modul 2_07

- Eine Query holt die Liste, jeder Zugriff auf die Beziehung löst eine weitere Query aus.
- Bei 500 Fahrzeugen sind das 501 Statements – in der Entwicklung unauffällig, in Produktion tödlich.
- Erkennung: mvn -Pinspect test und die Statements zählen.
- Vorhersage vor dem Lauf: Wie viele SELECTs erwarten Sie?

```
List<Vehicle> vehicles =  
    em.createQuery("SELECT v FROM Vehicle v", Vehicle.class).getResultList();  
for (Vehicle v : vehicles) {  
    System.out.println(v.getType().getModel().getBrand());    // je Runde 1 SELECT  
}  
  
-- Log:  
-- select ... from vehicle  
-- select ... from vehicletype where id=?      (x N)
```

Drei Wege aus dem N+1 – und wann welcher passt

Modul 2_07 – drei Lösungsmodule im Vergleich

Ansatz	Modul	Wirkung	Nachteil
FetchType.EAGER	2_07_..._Eager	lädt immer mit	gilt global, lädt auch dort, wo es niemand braucht
JOIN FETCH	2_07_..._FetchJoin	pro Query entschieden	Paging auf Collections wird unzuverlässig
@NamedEntityGraph	2_07_..._EntityGraph	wiederverwendbares Profil	mehr Schreibarbeit, Graph muss gepflegt werden

Regel: Mapping bleibt LAZY, die Query entscheidet – Fetch Join für den Einzelfall, Entity Graph für wiederkehrende Fälle.

Fetch Join und Entity Graph im Code

JOIN FETCH – pro Abfrage

```
List<Vehicle> l = em.createQuery(
    "SELECT v FROM Vehicle v " +
    "JOIN FETCH v.type t " +
    "JOIN FETCH t.model",
    Vehicle.class).getResultList();

// 1 Statement statt 1 + N
```

Entity Graph – wiederverwendbar

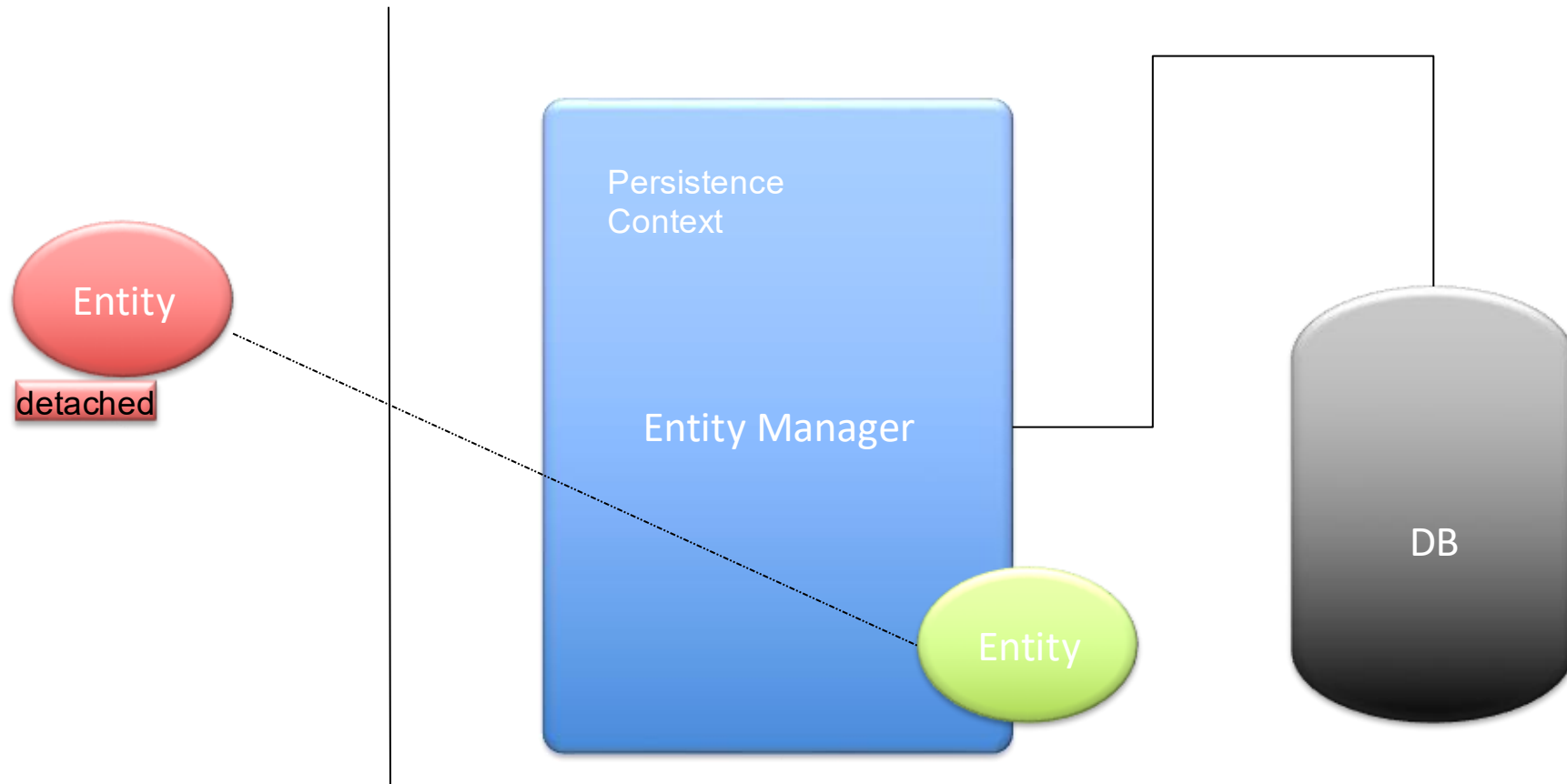
```
@NamedEntityGraph(name = "Vehicle.withType",
    attributeNodes = @NamedAttributeNode("type"))
@Entity public class Vehicle { ... }

em.createQuery(jpql, Vehicle.class)
    .setHint("jakarta.persistence.fetchgraph",
        em.getEntityGraph("Vehicle.withType"))
    .getResultList();
```

Beide lösen dasselbe Problem – der Entity Graph trennt die Ladestrategie von der Abfrage.

Detached Relations

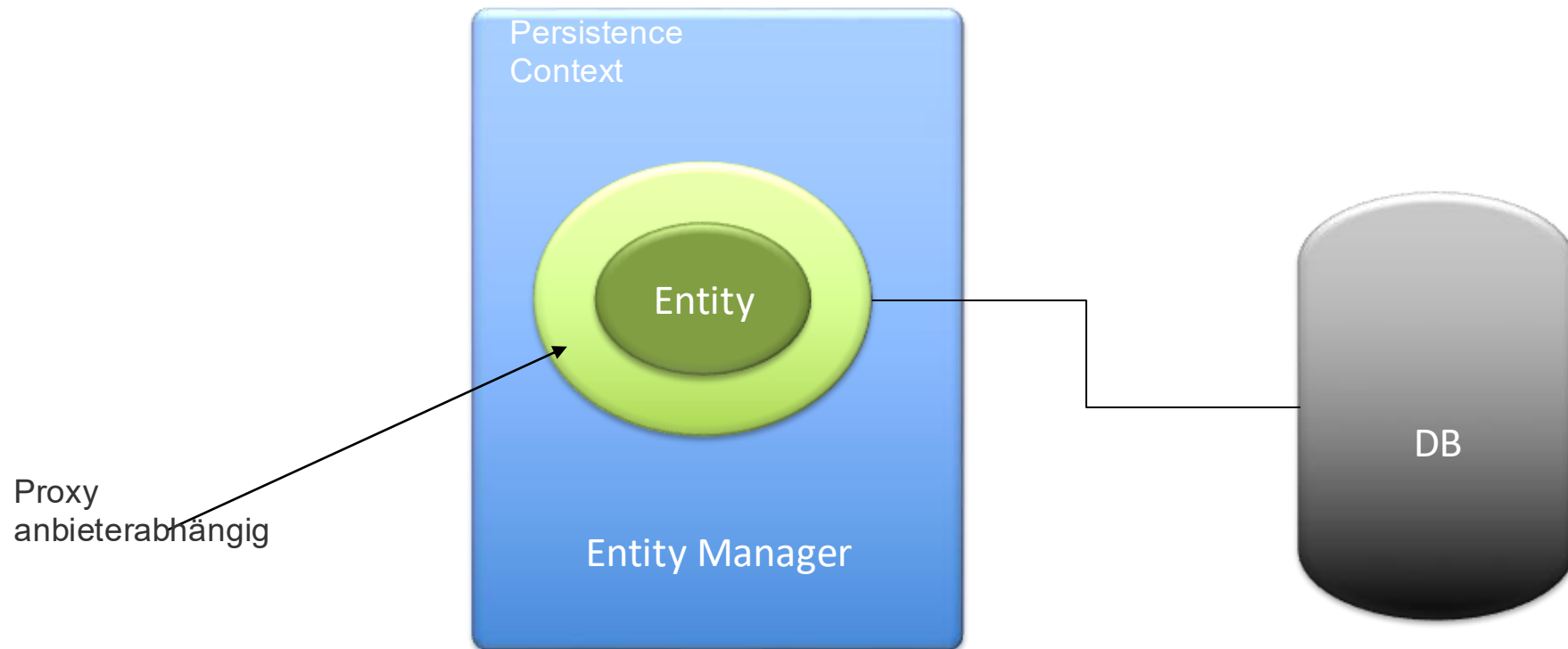
- Attached / Detached



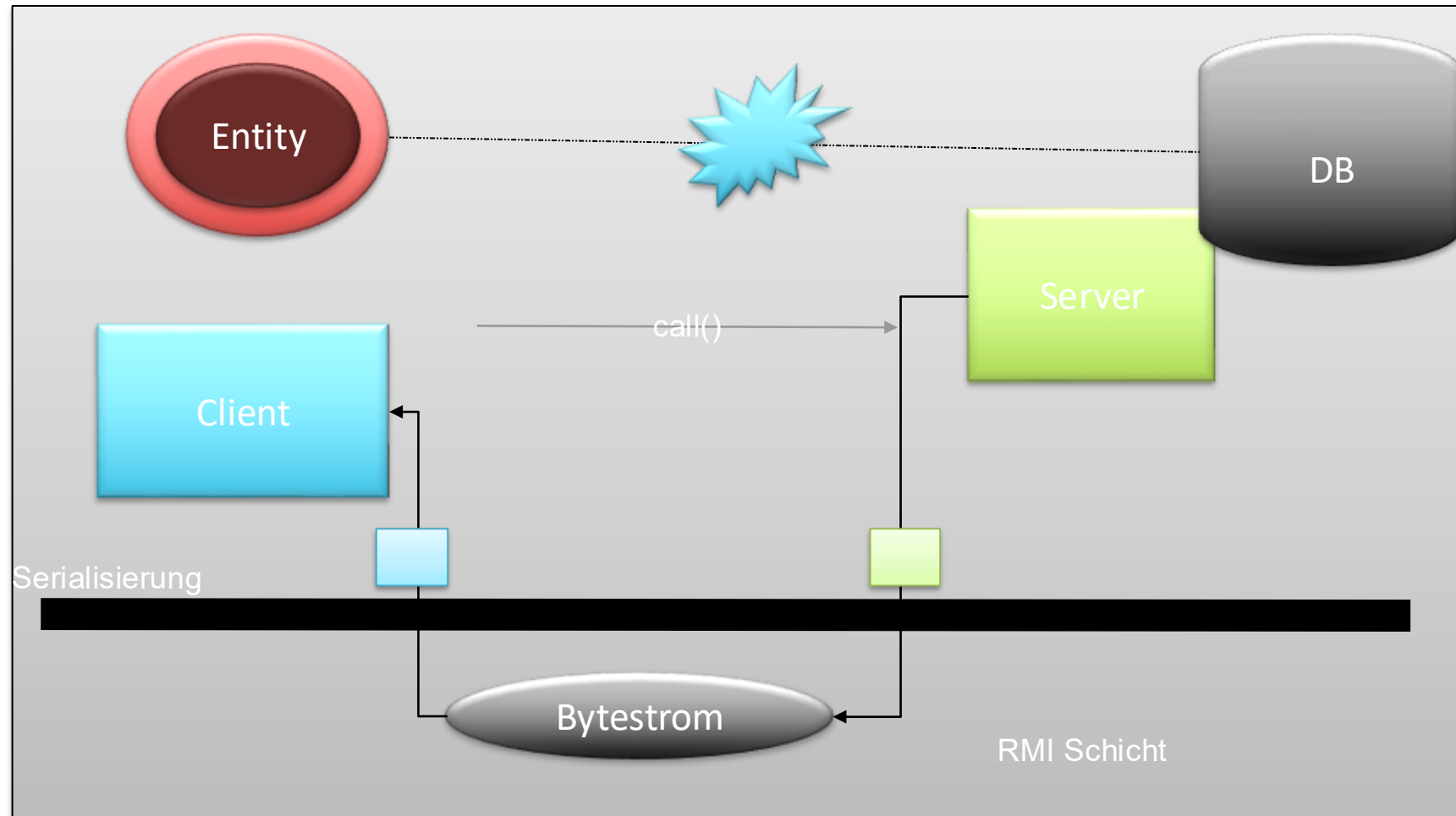
Entity Relationship

Lazy & Eager

- `@OneToMany (fetch=FetchType.EAGER)`



Detached Lazy



LazyInitializationException – und warum sie richtig liegt

Live-Coding: Fehler bewusst erzeugen, dann gemeinsam beheben

- Tritt auf, wenn eine Lazy-Beziehung nach dem Schließen des EntityManagers angefasst wird.
- Der Proxy hat keine Verbindung mehr – er kann nicht nachladen.
- Kein Fehler des Frameworks: die Transaktionsgrenze wurde überschritten.
- Lösungen: rechtzeitig per JOIN FETCH laden, DTO verwenden, oder die Transaktionsgrenze bewusst verschieben – nicht: open-session-in-view.

```
em.close();  
vehicle.getType().getModel();  
// org.hibernate.LazyInitializationException:  
//    could not initialize proxy - no Session
```

Übung: N+1 erkennen und beseitigen

🕒 25 min

Level ●●●

Modul 2_07

AUFTRAG

- Test mit -Pinspect laufen lassen und die SELECTs zählen.
- Vorher aufschreiben, wie viele Sie erwarten.
- Die Abfrage auf JOIN FETCH umstellen und erneut zählen.
- Variante mit Entity Graph bauen und beide vergleichen.

ERFOLGSKRITERIEN

- ✓ Statt 1 + N erscheint genau ein SELECT.
- ✓ Test grün, ohne das Mapping global auf EAGER zu stellen.
- ✓ Sie können begründen, welcher der drei Wege hier passt.

Zusatz für Schnelle: Was passiert bei JOIN FETCH auf eine Collection in Kombination mit setMaxResults? Ins Log schauen.

First Level, Second Level – und wer daran vorbeigeht

Warum ein UPDATE manchmal nicht ankommt

First Level Cache

Immer aktiv, pro EntityManager, nicht abschaltbar.

Garantiert Objektidentität innerhalb einer Transaktion.

Grundlage für Dirty Checking: der Provider kennt den geladenen Zustand.

Second Level Cache

Optional, providerspezifisch, über EntityManager hinweg geteilt.

@Cacheable plus shared-cache-mode in der persistence.xml.

Sinnvoll für Stammdaten, gefährlich bei häufigen Schreibzugriffen.

Am Cache vorbei

JPQL-Bulk-UPDATE und -DELETE.

Native Queries.

Folge: der Persistence Context liefert veraltete Werte – danach em.clear() aufrufen. Modul 2_08_05.

Ein JPQL-UPDATE läuft, danach liefert em.find den alten Wert. Warum?

- A) Die Transaktion wurde nicht committed.
- B) Der Bulk-Update umgeht den First-Level-Cache – die Instanz im Context ist unverändert.
- C) JPQL-UPDATE funktioniert nicht auf Entities mit @Version.
- D) Der Second-Level-Cache ist nicht konfiguriert.

Think – Pair – Share: 1 Minute allein, 1 Minute zu zweit, dann Auflösung.

Auflösung: Bulk Update und Cache

B – der Bulk-Update geht direkt an die Datenbank, am Persistence Context vorbei.

- Die bereits geladene Instanz kennt die Änderung nicht.
- Abhilfe: `em.clear()` nach dem Bulk-Update, oder den Update vor dem Laden ausführen.
- Gleiches gilt für Native Queries und für DELETE.
- Nebenwirkung beachten: `@Version` wird bei Bulk-Updates nicht automatisch hochgezählt.

Recap Kapitel 4

Dieselben Lernziele – jetzt als Häkchenliste.

- ✓ Sie kennen die Default-Fetch-Typen und deren Folgen.
- ✓ Sie erkennen ein N+1-Problem im Log und können es beziffern.
- ✓ Sie wählen zwischen Fetch Join und Entity Graph begründet.
- ✓ Sie erklären, warum Bulk-Operationen den Cache umgehen.

Als Nächstes: Korrektheit unter Last: Transaktionen, Locking und Lifecycle-Callbacks.

05

Transaktionen, Locking & Lifecycle

Nach diesem Kapitel können Sie ...

- die vier Entity-Zustände und ihre Übergänge sicher benennen,
- optimistisches und pessimistisches Sperren gegeneinander abwägen,
- Bean Validation und Lifecycle-Callbacks gezielt einsetzen,
- @Transactional in Spring auf das Gelernte übertragen.

Entity-Zustände

Modul 2_08_01

Zustand	Kennzeichen	Kommt hierhin durch	Wird gespeichert?
NEW	kein Bezug zum Context, keine ID	new	nein
MANAGED	im Persistence Context, überwacht	persist, find, Query	ja, automatisch beim Flush
DETACHED	hat ID, aber kein Context mehr	close, clear, detach	nein – erst nach merge
REMOVED	zum Löschen vorgemerkt	remove	DELETE beim Flush

persist macht aus NEW ein MANAGED – merge kopiert einen DETACHED-Zustand in eine verwaltete Instanz und gibt diese zurück.

Optimistisches Sperren mit @Version

Modul 2_08_02

- Kein Lock in der Datenbank: es wird beim Schreiben geprüft, nicht beim Lesen.
- Das UPDATE enthält die Versionsnummer in der WHERE-Klausel.
- Trifft es keine Zeile, gewann ein anderer – OptimisticLockException.
- Richtige Reaktion: neu laden, fachlich zusammenführen, Anwender informieren – kein blindes Retry in einer Schleife.

```
@Version
private int version;

-- erzeugtes SQL:
update vehicle set price=?, version=6 where id=? and version=5
```

Pessimistisches Sperren

Modul 2_08_06

- Sperre in der Datenbank, solange die Transaktion läuft.
- PESSIMISTIC_READ erlaubt Lesen, PESSIMISTIC_WRITE sperrt exklusiv.
- Sichtbar im Log als SELECT ... FOR UPDATE.
- Nur für kurze, klar begrenzte Abschnitte – sonst Deadlocks und Wartezeiten.

```
Vehicle v = em.find(Vehicle.class, id, LockModeType.PESSIMISTIC_WRITE);
```

```
-- Log:  
-- select ... from vehicle where id=? for update
```

Bean Validation und equals/hashCode

Bean Validation – Modul 2_08_03

```
@NotBlank
private String licensePlate;

@Min(1)
private int seats;

// geprüft automatisch bei
// pre-persist und pre-update
// -> ConstraintViolationException
```

equals/hashCode – Modul 2_08_04

```
// falsch: ID als Basis
// vor dem persist ist sie null

// besser: fachlicher Schlüssel
public boolean equals(Object o) { ...
    return Objects.equals(
        licensePlate, other.licensePlate); }
public int hashCode() {
    return Objects.hash(licensePlate); }
```

Der Klassiker: Entity vor dem persist in ein HashSet legen, danach wird sie dort nicht mehr gefunden – weil sich der Hash mit der ID ändert.

Lifecycle-Callbacks und Listener

Module 2_12, 2_13, 2_14

- Callbacks direkt an der Entity: @PrePersist, @PostPersist, @PreUpdate, @PostLoad, @PreRemove (Modul 2_12).
- Ausgelagert in eine eigene Klasse über @EntityListeners (Modul 2_13).
- Für alle Entities zentral als Default-Listener in der orm.xml (Modul 2_14).
- Typischer Einsatz: Auditing von Erstell- und Änderungszeitpunkt.
- Regel: im Callback keine weiteren Entities ändern und keinen EntityManager verwenden.

```
@EntityListeners(VehicleTypeListener.class)
@Entity
public class VehicleType { ... }

public class VehicleTypeListener {
    @PrePersist
    void onCreate(VehicleType t) { t.setCreated(LocalDateTime.now()); }
}
```

Transaktions-Propagation auf einen Blick

Ersetzt sechs Einzeldiagramme – dieselbe Information, ein Bild

Attribut	Mit laufender Transaktion	Ohne Transaktion	Typischer Einsatz
REQUIRED	läuft darin mit	startet eine neue	Standardfall
REQUIRES_NEW	setzt aus, startet eigene	startet eine neue	Protokollierung, die überleben soll
SUPPORTS	läuft mit	läuft ohne	reine Leseoperationen
NOT_SUPPORTED	setzt aus	läuft ohne	lange Reports
MANDATORY	läuft mit	Fehler	interne Hilfsmethoden
NEVER	Fehler	läuft ohne	explizit transaktionsfreier Code

Brücke: und in Spring?

Dieselben Konzepte, andere Syntax – die Welt, in der Sie arbeiten

- @Transactional entspricht REQUIRED als Default.
- Die Proxy-Falle: ein Selbstaufruf innerhalb derselben Bohne geht am Proxy vorbei – die Annotation wirkt nicht.
- readOnly = true setzt die Hibernate-Session auf FlushMode.MANUAL: kein Dirty Checking, spürbar bei großen Leselisten.
- Rollback erfolgt standardmäßig nur bei RuntimeException – checked exceptions müssen mit rollbackFor angegeben werden.

```
@Transactional
public void book() {
    this.audit();      // Selbstaufruf: kein neues Transaktion!
}

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void audit() { ... }
```

Recap Kapitel 5

Dieselben Lernziele – jetzt als Häkchenliste.

- ✓ Sie benennen die vier Entity-Zustände und ihre Übergänge.
- ✓ Sie wählen zwischen optimistischem und pessimistischem Sperren begründet.
- ✓ Sie setzen Validierung und Callbacks an der richtigen Stelle ein.
- ✓ Sie übertragen das Gelernte auf `@Transactional` in Spring.

Als Nächstes: Abfragen programmatisch bauen – die Criteria API.

06

Criteria API & Modernisierung

Nach diesem Kapitel können Sie ...

- eine einfache Criteria-Abfrage aufbauen und ausführen,
- einschätzen, wann Criteria gegenüber JPQL die bessere Wahl ist,
- moderne Typen aus `java.time` korrekt abbilden.

Criteria API – das Grundgerüst

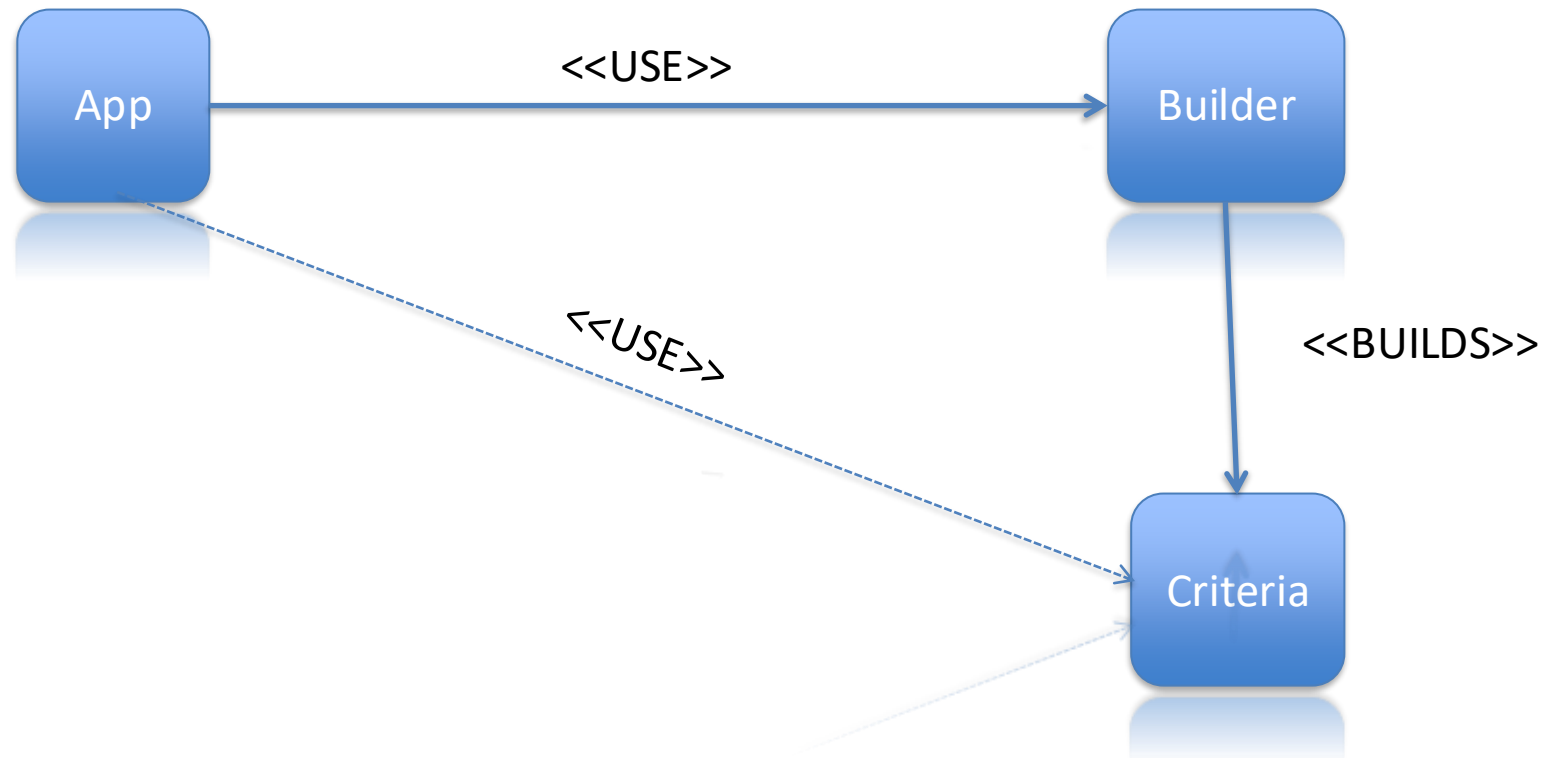
Modul 2_15

- Vier Schritte: Builder holen, Query erzeugen, Root setzen, Bedingungen anhängen.
- Stärke: dynamisch zusammengesetzte Filter – kein String-Basteln.
- Schwäche: deutlich länger und schwerer lesbar als JPQL.
- Mit dem Metamodel (`Vehicle_.licensePlate`) wird es typsicher.

```
CriteriaBuilder cb = em.getCriteriaBuilder();
CriteriaQuery<Vehicle> cq = cb.createQuery(Vehicle.class);
Root<Vehicle> v = cq.from(Vehicle.class);
cq.select(v).where(cb.like(v.get("licensePlate"), "M-%"));

List<Vehicle> l = em.createQuery(cq).getResultList();
```

CRITERIA API



```
CriteriaQuery<Basket> queryBaskets = criteriaBuilder.createQuery(Basket.class);
```

```
Root<Basket> baskets= queryBaskets.from(Basket.class);  
queryBaskets.select(baskets);
```

```
List<Basket> resultList = manager.createQuery(queryBaskets).getResultList();
```

```
//SELECT b from Basket b
```

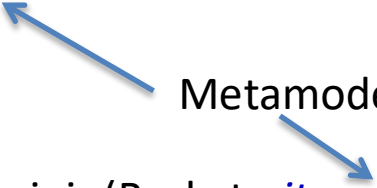
JPQL ist kompakter, Criteria API ist **typsicher und dynamisch**.

Aufbau einer Query

```
Path<BigDecimal> priceAttribute = root.get(Basket_.price)  
//without Metamodel root.get("price")
```

```
SetJoin<Basket, ArticleItem> itemsOfBasket=    baskets.join(Basket_.items);
```

Metamodel



- Typsicherheit
- Autocomplete
- Weniger Fehler
- Annotation Processing.

Metamodel

```
Path<String> nameAttribute = itemsOfBasket.get(ArticleItem_.name)
```

```
Predicate nameEqualsIPAD = criteriaBuilder.equal(nameAttribute, "IPad");
```

```
queryBaskets.select(baskets).where(nameEqualsIPAD).  
    distinct(true);
```

WHERE

```
CriteriaQuery<Basket> queryBaskets =  
criteriaBuilder.createQuery(Basket.class);
```

```
Root<Basket> root= queryBaskets.from(Basket.class);
```

```
queryBaskets.select(root)  
    .where(builder.gt(root.get("price"), 100));
```

```
SELECT root FROM Basket root WHERE root.price > 100
```

Takeway: JPQL ist kompakter, Criteria API ist **typsicher und dynamisch**.

JPQL vs Criteria

```
Path<String> nameAttribute = itemsOfBasket.get(ArticleItem_.name)
```

```
Predicate nameEqualsIPAD = criteriaBuilder.equal(nameAttribute, "IPad");
```

```
queryBaskets.select(baskets).where(nameEqualsIPAD).  
    distinct(true);
```

- cb.and(...)
- cb.or(...)
- cb.not(...)

WHERE

Kombination

```
CriteriaQuery<Person> query = cb.createQuery(Person.class);
```

```
Root<Person> root = query.from(Person.class);
```

```
Predicate olderThan18 = cb.gt(root.get("age"), 18);
```

```
Predicate nameStartsWithA = cb.like(root.get("name"), "A%");
```

```
query.where(cb.and(olderThan18, nameStartsWithA));
```

- cb.and(...)
- cb.or(...)
- cb.not(...)

Aggregation

```
CriteriaQuery<Long> query =  
cb.createQuery(Long.class);  
...  
  
query.select(cb.count(root))  
    .where(cb.and(older18 , nameWithA));
```

- count, sum, avg, min, max
- groupBy, having

Typische Fehler

- Vergessen, `select(root)` zu setzen
- Falsche Property-Namen bei `root.get("...")`
- Zu komplexe Queries → lieber QueryDSL oder Spring Data Specifications
- Metamodel nicht generiert (Annotation Processing fehlt)



Wann CriteriaAPI

Gut geeignet für:

- Dynamische Filter
- Komplexe, programmatisch erzeugte Queries
- Frameworks (z. B. Spring Data Specifications)



Weniger geeignet für:

- Einfache statische Queries → JPQL ist lesbarer
- Sehr komplexe Query-Logik → QueryDSL oft eleganter

Aufgabe 2_15: Überblick

- Du findest im Modul `2_15_CriteriaAPI_Simple` die Testklasse `net.rentacar.TestQuery`. Diese enthält drei leere Testmethoden, die du mit Hilfe der Criteria API implementieren sollst. Die fachlichen Anforderungen entsprechen denen aus der Übung `2_04_JPQL_WhereClause`, jedoch soll hier kein JPQL-String, sondern das Criteria-Objektmodell verwendet werden.



Aufgabe 2_15: Einzelaufgabe

- `testQueryForNutzerWithPersonnameISMichaelOrMathias`
 - Frage alle User ab, deren zugehörige Person den Vornamen "Michael" oder "Mathias" hat.
 - Verwende den `in()` Operator auf dem Attributpfad `person.firstName`.
 - **Erwartung:** 2 Ergebnisse.
- `testQueryForShopMitKeinemVehicle`
 - Finde alle Shop-Objekte, die aktuell kein Fahrzeug im Fuhrpark (carpool) haben.
 - Nutze dazu die Methode `cb.isEmpty()` des `CriteriaBuilder`.
 - **Erwartung:** 2 Ergebnisse.
- `testQueryForShopsichtInStuttgartUndMuenchen_MitINOperator`
 - Suche alle Shops, deren Standort (location) **nicht** in "Muenchen" oder "Stuttgart" liegt.
 - Verwende eine Kombination aus `cb.not()` und dem `in()` Operator auf dem Feld `location`.
 - **Erwartung:** 1 Ergebnis.
- **Tipps**
 - Nutze `manager.getCriteriaBuilder()`, um den Builder zu erhalten.
 - Erstelle eine `CriteriaQuery<T>` für den gewünschten Ergebnistyp.
 - Definiere den Root-Pfad mit `cq.from(Entity.class)`.
 - Denke daran, die Abfrage mit `manager.createQuery(cq).getResultList()` auszuführen.





Zusatz

- `testQueryForUserByLastName`
 - Suche nach dem User, dessen Person den Nachnamen "Mayer" trägt.
 - Nutze `user.get("person").get("lastName")`.
 - **Erwartung:** 1 Ergebnis.
- `testCountAllShops`
 - Ermittle die Gesamtzahl aller registrierten Shops.
 - Verwende `cb.count()` und als Ergebnistyp der Query `Long.class`.
 - **Erwartung:** 3 Ergebnisse (Long-Wert 3).

Zusammenfassung



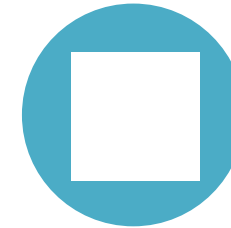
**TYPESICHERE,
DYNAMISCHE
QUERY-GENERIERUNG**



FÜR FLEXIBLE FILTER



**ETWAS VERBALER ALS
JPQL**



**METAMODEL STEIGERT
TYPESICHERHEIT UND
IDE-SUPPORT**

Datum und Zeit: java.time statt java.util.Date

Legacy

```
@Temporal(TemporalType.DATE)
private Date created;

@Temporal(TemporalType.TIMESTAMP)
private Calendar lastModified;

// veränderlich, nicht threadsicher,
// Zeitzone-Fallen
```

Seit JPA 2.2

```
private LocalDate created;
private LocalDateTime lastModified;
private Instant importedAt;

// kein @Temporal nötig
// unveränderlich und eindeutig
```

@Temporal nur noch für Altbestand – in neuem Code sind die Typen aus java.time der Standard.

Zum Abschluss

Was Sie mitnehmen und wo es weitergeht

- Exit-Ticket: eine Erkenntnis, eine offene Frage – bitte auf die Karte.
- Das Cheat Sheet begleitet Sie weiter – es deckt alle 40 Module ab.
- Wiederholung mit Wirkung: dasselbe Modell noch einmal mit einer anderen Vererbungsstrategie abbilden und das SQL vergleichen.
- Nächste sinnvolle Schritte: Spring Data JPA, Flyway oder Liquibase für Schema-Migrationen, Testcontainers statt In-Memory-Datenbank.
- Alle Lösungsmodule bleiben im Repository – inklusive der drei Varianten zum N+1-Problem.